You said:

```
JASC-PAL
0100
256
0 0 0
15 15 15
31 31 31
47 47 47
63 63 63
75 75 75
91 91 91
107 107 107
123 123 123
139 139 139
155 155 155
171 171 171
187 187 187
203 203 203
219 219 219
235 235 235
15 11 7
23 15 11
31 23 11
39 27 15
47 35 19
55 43 23
63 47 23
75 55 27
83 59 27
91 67 31
99 75 31
107 83 31
115 87 31
123 95 35
131 103 35
143 111 35
11 11 15
19 19 27
27 27 39
39 39 51
47 47 63
55 55 75
63 63 87
71 71 103
79 79 115
91 91 127
99 99 139
107 107 151
115 115 163
123 123 175
131 131 187
139 139 203
0 0 0
7 7 0
11 11 0
19 19 0
27 27 0
35 35 0
43 43 7
47 47 7
55 55 7
63 63 7
```

71 71 7
75 75 11
83 83 11
91 91 11
99 99 11
107 107 15
7 0 0
15 0 0
23 0 0
31 0 0
39 0 0
47 0 0
55 0 0
63 0 0
71 0 0
79 0 0
87 0 0
95 0 0
103 0 0
111 0 0
119 0 0
127 0 0
19 19 0
27 27 0
35 35 0
47 43 0
55 47 0
67 55 0
75 59 7
87 67 7
95 71 7
107 75 11
119 83 15
131 87 19
139 91 19
151 95 27
163 99 31
175 103 35
35 19 7
47 23 11
59 31 15
75 35 19
87 43 23
99 47 31
115 55 35
127 59 43
143 67 51
159 79 51
175 99 47
191 119 47
207 143 43
223 171 39
239 203 31
255 243 27
11 7 0
27 19 0
43 35 15
55 43 19
71 51 27
83 55 35
99 63 43
111 71 51
127 83 63

139 95 71
155 107 83
167 123 95
183 135 107
195 147 123
211 163 139
227 179 151
171 139 163
159 127 151
147 115 135
139 103 123
127 91 111
119 83 99
107 75 87
95 63 75
87 55 67
75 47 55
67 39 47
55 31 35
43 23 27
35 19 19
23 11 11
15 7 7
187 115 159
175 107 143
163 95 131
151 87 119
139 79 107
127 75 95
115 67 83
107 59 75
95 51 63
83 43 55
71 35 43
59 31 35
47 23 27
35 19 19
23 11 11
15 7 7
219 195 187
203 179 167
191 163 155
175 151 139
163 135 123
151 123 111
135 111 95
123 99 83
107 87 71
95 75 59
83 63 51
67 51 39
55 43 31
39 31 23
27 19 15
15 11 7
111 131 123
103 123 111
95 115 103
87 107 95
79 99 87
71 91 79
63 83 71
55 75 63

47 67 55
43 59 47
35 51 39
31 43 31
23 35 23
15 27 19
11 19 11
7 11 7
255 243 27
239 223 23
219 203 19
203 183 15
187 167 15
171 151 11
155 131 7
139 115 7
123 99 7
107 83 0
91 71 0
75 55 0
59 43 0
43 31 0
27 15 0
11 7 0
0 0 255
11 11 239
19 19 223
27 27 207
35 35 191
43 43 175
47 47 159
47 47 143
47 47 127
47 47 111
47 47 95
43 43 79
35 35 63
27 27 47
19 19 31
11 11 15
43 0 0
59 0 0
75 7 0
95 7 0
111 15 0
127 23 7
147 31 7
163 39 11
183 51 15
195 75 27
207 99 43
219 127 59
227 151 79
231 171 95
239 191 119
247 211 139
167 123 59
183 155 55
199 195 55
231 227 87
127 191 255
171 231 255
215 255 255

103 0 0
139 0 0
179 0 0
215 0 0
255 0 0
255 243 147
255 247 199
255 255 255
159 91 83

this is the palette of quake.
tell me how many differen hues are represented and wich ones.

ChatGPT said:

The Quake palette contains **8 major chromatic hue families**, plus a substantial range of achromatic (grayscale) colors.

If we classify the colors by their position on the traditional color wheel, the palette contains:

Hue family	Colors represented
1. Red	Dark red, blood red, bright red
2. Orange	Orange, burnt orange, rust
3. Yellow	Yellow, golden yellow, ochre
4. Green	Olive, yellow-green, natural green
5. Cyan	Blue-green, turquoise, cyan
6. Blue	Dark blue, royal blue, light blue
7. Violet	Purple, violet, blue-violet
8. Magenta	Magenta, pink, reddish purple

Additionally:

- **Brown:** Several shades, primarily dark and desaturated orange hues.
- **Grayscale:** Black, white, and intermediate grays. These have no chromatic hue.

Important distinction

There are two ways to count hues:

1. **Color families:** 8 major hue families, as listed above.
2. **Mathematical HSV hue values:** Potentially many distinct hue angles, depending on rounding and the treatment of near-grayscale colors.

For understanding and organizing the Quake palette, I would use the 8-family classification, with brown and grayscale treated as additional color groups rather than independent hues.

You said:

i want the mathematical ones. convert them to hsv and compare the hues.

ChatGPT said:

For a mathematical comparison, I'll use standard HSV (Hue, Saturation, Value), with:

- **Hue:** 0°–360°
- **Saturation:** 0–100%
- **Value:** 0–100%

I'll exclude achromatic entries (black, white and grays) from the hue count because their saturation is zero and their hue is undefined.

Importantly, I'll distinguish between:

1. **Unique hue angles**: exact distinct HSV hue values.
2. **Hue distribution**: how many palette colors fall at each hue angle.
3. **Hue clusters**: nearby angles grouped together to identify the actual color ranges.

One important detail: exact hue angles can differ by fractions of a degree even when two colors look virtually identical. I would therefore report both the exact unique angles and a second classification grouped into 5° hue intervals.

The resulting analysis will tell us how many mathematically distinct hues Quake actually uses, rather than imposing conventional color names on the palette.

You said:

sounds good do it

Worked for 23s

Analyzed

Analyzed

Analyzed

Analyzed

ChatGPT said:

I calculated the HSV values for all 256 RGB entries in your Quake palette.

110 distinct hues

Out of 256 palette entries · Standard HSV · Hue in degrees

1. Results

Measurement	Result
Total palette entries	256
Achromatic entries (S = 0)	18
Chromatic entries	238
Unique RGB colors	244
Distinct hue angles	110

Hue angles are calculated from the RGB values using standard HSV conversion, with angles rounded to 2 decimal places for presentation.

2. All distinct hue angles

Here are all 110 unique hue values, sorted numerically.

Red / orange / yellow region (0°–60°)

```
0.00
4.42
5.60
6.32
8.00
8.11
10.29
10.43
11.05
11.43
12.86
13.33
```

```
14.12
15.00
15.56
17.14
18.00
18.75
20.00
20.49
21.18
21.43
21.82
22.11
22.50
23.33
24.00
24.38
25.00
25.50
25.71
26.67
29.14
29.19
30.00
30.91
32.73
32.90
33.33
33.53
34.29
35.00
35.56
36.00
36.43
36.59
37.50
38.18
38.82
39.23
40.00
40.91
41.05
42.22
42.50
42.86
43.04
43.26
43.64
43.73
44.00
45.00
45.88
46.54
46.81
46.87
47.59
49.09
49.25
49.62
50.27
51.27
51.43
52.50
53.02
```

```
53.33
53.62
54.89
55.20
55.56
56.84
58.33
60.00
```

Green region (120°–156°)

```
120.00
135.00
140.00
144.00
156.00
```

Cyan region (180°–210°)

```
180.00
197.14
210.00
```

Blue region

```
240.00
```

Purple / magenta region (315°–352°)

```
315.00
322.50
323.33
326.67
328.24
330.00
332.00
333.33
336.92
337.50
340.00
342.00
342.86
343.64
346.67
348.00
350.00
351.43
```

3. Distribution in 5° intervals

This is particularly interesting because it reveals how Quake distributes its palette across the color wheel.

4. What this tells us

There are some very pronounced characteristics:

- **Red/orange/yellow:** A very dense range from 0° to 60°.
- **Green:** Relatively sparse, with concentrations around 120° and 144°.
- **Cyan:** Almost entirely absent.
- **Blue:** A very strong concentration at exactly 240°.
- **Purple/magenta:** A relatively narrow range from 315° to 351°.

The palette is therefore not evenly distributed around the HSV hue circle. It is heavily weighted toward warm colors, with a substantial blue component.

One technical note: The 5° histogram counts palette entries, not unique hue angles. Also, hue is undefined for grayscale colors, so those 18 entries are excluded.

If you're using this for palette analysis or procedural color matching, the next useful step would be to calculate the actual hue clusters, using circular distance and a clustering algorithm, rather than relying on arbitrary 5° boundaries.

‹/›

You said:

now list the diferent hue families and how they internally vary in hue

ChatGPT said:

The Quake palette has a very uneven hue distribution. Its chromatic colors form several distinct hue families, with some families having considerable internal hue variation and others being concentrated around a single angle.

Using conventional HSV hue sectors, here is the breakdown based on the calculated angles.

1. Hue families and their internal variation

Hue family	Hue interval	Quake hue values
Red	345°–15°	0°, 4.42°, 5.60°, 6.32°, 8°, 8.11°, 10.29°, 10.43°, 11.05°, 11.43°, 12.86°, 13.33°, 14.12°
Orange	15°–45°	15°, 15.56°, 17.14°, 18°, 18.75°, 20°, 20.49°, 21.18°, 21.43°, 21.82°, 22.11°, 22.50°, 23.33°, 24°, 24.38°, 25°, 25.50°, 25.71°, 26.67°, 29.14°, 29.19°, 30°, 30.91°, 32.73°, 32.90°, 33.33°, 33.53°, 34.29°, 35°, 35.56°, 36°, 36.43°, 36.59°, 37.50°, 38.18°, 38.82°, 39.23°, 40°, 40.91°, 41.05°, 42.22°, 42.50°, 42.86°, 43.04°, 43.26°, 43.64°, 43.73°, 44°
Yellow	45°–75°	45°, 45.88°, 46.54°, 46.81°, 46.87°, 47.59°, 49.09°, 49.25°, 49.62°, 50.27°, 51.27°, 51.43°, 52.50°, 53.02°, 53.33°, 53.62°, 54.89°, 55.20°, 55.56°, 56.84°, 58.33°, 60°
Green	75°–165°	120°, 135°, 140°, 144°, 156°
Cyan	165°–195°	180°

Blue	195°–255°	197.14°, 210°, 240°
Violet	255°–285°	None
Magenta	285°–345°	315°, 322.50°, 323.33°, 326.67°, 328.24°, 330°, 332°, 333.33°, 336.92°, 337.50°, 340°, 342°, 342.86°, 343.64°

Note: 346.67°, 348°, 350° and 351.43° belong to the red family, completing its wraparound range.

2. Visual representation of the hue distribution

Red
30° across 0°

Orange
15°–45°

Yellow
45°–60°

Green
120°–156°

Cyan
180°

Blue
197°–240°

Violet
No samples

Magenta
315°–343.64°

3. The most interesting characteristics

Three observations stand out:

- **Orange is particularly diverse:** it has a large number of distinct hue angles, making it one of the most nuanced color families in the palette.
- **Green is discontinuous:** there is a large gap between yellow and the first green hues, and relatively few green hue angles.
- **Blue and magenta are highly selective:** the blue region has a strong concentration at 240°, while magenta occupies a relatively narrow range.

4. A useful distinction for further analysis

Hue alone does not describe all the variation in the Quake palette. Two colors can have exactly the same hue but very different saturation and brightness.

For a complete palette analysis, I would separate each family into three dimensions:

- **Hue variation:** changes in the actual color angle.
- **Saturation variation:** changes from muted to vivid.
- **Value variation:** changes from dark to bright.

This would let us identify the actual color-generation structure of the Quake palette, rather than just its hue distribution.

One caveat: the numerical list above inherits the rounded hue results from the preceding calculation. The family boundaries are conventional HSV sectors, not perceptually optimized clusters.

You said:

What i am looking for is a more intelligent algorythm for reducing images to quake palette. My naive approach would be to look for colors that are underrepresented in the palette and decide where to shift them.

ChatGPT said:

Your approach is on the right track, but I would make one important change: **don't redistribute colors simply according to which hues are underrepresented. Redistribute them according to the image's visual content and the palette's ability to represent that content.**

The problem is essentially a constrained color-quantization problem, but with an additional requirement: preserve the visual character of Quake's palette.

I would build a perceptually weighted, adaptive palette-mapping algorithm.

1. Why simple nearest-color matching is insufficient

Suppose an image contains a large area of turquoise (hue 185°), but the Quake palette has very few turquoise colors.

A conventional nearest-color algorithm might map it to:

- Cyan, if available.
- Blue, if closer in RGB distance.
- Green, if the saturation and brightness make that closer.

The result can be a substantial change in the image's color identity.

However, blindly shifting every turquoise toward cyan would also be wrong. The correct mapping depends on:

- The original hue.
- Saturation.
- Brightness.
- The colors surrounding it.
- The amount of image area represented by that color.
- The available palette colors in that brightness range.

2. My proposed algorithm

I would use five stages.

1. **Analyze the source image**

   Build a color histogram in a perceptual color space, preferably OKLab or CIELAB.

2. **Analyze the Quake palette**

   Build a multidimensional representation of available colors, including their hue, chroma and lightness.

3. **Identify palette deficiencies**

   Find source colors that cannot be represented well, taking their image frequency and visual importance into account.

4. **Calculate adaptive color transformations**

   Determine where missing colors should move, preserving hue relationships and tonal structure as much as possible.

5. **Quantize to the actual palette**

   Assign each pixel to an existing Quake palette entry, optionally using dithering.

3. The key: measure palette coverage

I would not measure hue coverage independently of brightness and saturation.

For example, Quake might have plenty of blue hues, but almost all of them might be dark or highly saturated.

A light, desaturated blue could still be poorly represented.

Instead, create a three-dimensional histogram:

- Hue: 360 bins (circular).
- Saturation/chroma: 32 bins.
- Lightness/value: 32 bins.

For every source color, measure the distance to the nearest palette color.

This gives you a coverage map of the palette.

4. A better color-distance metric

I recommend **OKLab / OKLCH**, rather than HSV.

HSV is useful for describing hue, but it is not perceptually uniform. A 10° hue difference does not represent the same perceived difference everywhere on the color wheel.

Convert both source colors and palette colors into OKLCH:

- (L): perceptual lightness.
- (C): chroma (colorfulness).
- (h): hue angle.

Then calculate a distance such as Euclidean OKLab distance:

[
d(c,p)=\sqrt{
(L_c-L_p)^2+
(a_c-a_p)^2+
(b_c-b_p)^2
}
]

This automatically handles the fact that hue differences matter less for nearly gray colors.

For example, a low-chroma blue-gray should not be penalized heavily for having a slightly different hue from its nearest palette equivalent.

5. Build a palette coverage map

For each source color, find its nearest Quake palette color and record the error.

I would measure three things:

Metric	What it tells you
Mean perceptual error	Overall quality of the conversion
Maximum perceptual error	Worst-represented colors
Weighted error	Which mistakes affect the most pixels

The weighted error is particularly important.

A color appearing in 20% of an image should have considerably more influence on the optimization than a color appearing in 0.01%.

You can also visualize the palette's deficiencies as a heatmap in OKLCH space.

6. Adaptive hue redistribution

This is where your original idea becomes interesting.

Imagine the source image contains these colors:

Source hue	Image coverage

10°	12%
25°	8%
45°	5%
90°	15%
180°	25%
210°	20%
280°	15%

Suppose the palette has excellent coverage around 10°–60°, but poor coverage around 180°–210°.

Instead of shifting everything by a fixed amount, calculate a hue transformation:

[
h'=f(h)
]

The function (f) should have these properties:

1. Preserve hue where the palette already represents colors well.
2. Compress hue ranges where the palette has insufficient coverage.
3. Expand ranges where additional palette colors are available.
4. Preserve the ordering of hues.
5. Avoid discontinuities at the 0°/360° boundary.

This is essentially a nonlinear, circular hue remapping function.

Important: Do this independently for different lightness and chroma ranges. A hue transformation that works for bright saturated colors may be completely inappropriate for dark, muted colors.

7. Don't redistribute according to palette frequency alone

There is a subtle but important distinction.

Suppose Quake has 30 red entries and only 3 cyan entries.

That doesn't necessarily mean cyan should be shifted toward red.

Those 30 red entries might be almost identical in hue but distributed across many brightness levels.

The relevant question is:

How well does the palette cover the colors in the source image?

Not:

How many palette entries does each hue family contain?

This distinction will make a considerable difference in the quality of the algorithm.

8. The optimization objective

You can formalize the problem as minimizing:

[
E =
E_{\text{perceptual}}+
\lambda_1 E_{\text{hue}}+

$$
\lambda_2 E_{\text{structure}}+
\lambda_3 E_{\text{transformation}}
$$

Where:

- $E_{\text{perceptual}}$: Difference between original pixels and mapped palette colors.
- E_{hue}: Penalty for undesirable hue changes.
- $E_{\text{structure}}$: Penalty for destroying local color relationships.
- $E_{\text{transformation}}$: Penalty for excessive or abrupt color remapping.

The weights (λ) determine how much importance each property receives.

I would initially prioritize perceptual accuracy, then add the other constraints.

9. Implementation strategy

I would develop this incrementally:

Version	Algorithm	Purpose
V1	Nearest palette color in OKLab	Baseline
V2	Image-frequency-weighted color mapping	Improve important colors
V3	Adaptive hue redistribution	Compensate for palette deficiencies
V4	Lightness/chroma-dependent hue mapping	Preserve tonal relationships
V5	Spatially aware mapping + dithering	Improve visual coherence

One particularly interesting technique for V3 is *optimal transport*. It can help construct a smooth mapping between the source image's color distribution and the available palette distribution. However, I would constrain it so that it doesn't blindly force the two histograms to match.

10. One important design decision

There are two different goals you could optimize for:

A. Faithful conversion

Preserve the original image's colors as closely as possible. Accept that some colors will be poorly represented by Quake's palette.

Best for photographic images

B. Quake-style conversion

Deliberately transform colors to fit the palette's characteristic color distribution. Accept larger perceptual differences if the resulting image looks more coherent and authentic to Quake.

Best for stylized game textures

My recommendation would be to support both as a continuous parameter, rather than implementing two separate algorithms.

One question that will significantly influence the design:

Are you primarily converting photographs and illustrations into Quake textures, or converting existing game textures and artwork into the Quake aesthetic?

The distinction matters because the second case can benefit from much more aggressive hue redistribution.

You said:

ok! Give me a python script converting an input image to a quake palettized one with the method you suggest.

Worked for 19s

ChatGPT said:

I'll implement this as a command-line Python tool with the following characteristics:

- Reads your original **JASC-PAL Quake palette** directly.
- Converts RGB colors into **OKLab** for perceptually better color matching.
- Finds the nearest Quake palette color using perceptual distance, not RGB distance or HSV distance.
- Supports Floyd–Steinberg dithering.
- Produces a genuine indexed PNG (8-bit palette image), not an RGB image that merely looks palettized.
- Preserves the original image dimensions.

I'll make the palette an external `.pal` file so you can experiment with other palettes without changing the code.

1. Python script

Save as `quake_quantize.py`.

```
#!/usr/bin/env python3

"""
Quake palette image converter.

Features:
- JASC-PAL palette loading
- OKLab perceptual color matching
- Optional Floyd-Steinberg dithering
- Indexed PNG output (8-bit, 256-color palette)
"""

import argparse
import numpy as np
from PIL import Image
from scipy.spatial import cKDTree


# ------------------------------------------------------------
# JASC-PAL LOADER
# ------------------------------------------------------------

def load_palette(filename):
    with open(filename, "r") as f:
        lines = [line.strip() for line in f if line.strip()]

    if lines[0] != "JASC-PAL":
        raise ValueError("Not a JASC-PAL file")

    if lines[1] != "0100":
        raise ValueError("Unsupported JASC-PAL version")

    count = int(lines[2])

    colors = []
    for line in lines[3:]:
        rgb = [int(v) for v in line.split()]
        if len(rgb) != 3 or any(v < 0 or v > 255 for v in rgb):
            raise ValueError(f"Invalid palette entry: {line}")
        colors.append(rgb)

    if len(colors) != count:
```

```
            raise ValueError(
                f"Expected {count} colors, found {len(colors)}"
            )

        return np.array(colors, dtype=np.uint8)


    # ------------------------------------------------------------
    # RGB -> OKLAB
    # ------------------------------------------------------------

    def rgb_to_oklab(rgb):
        """
        Convert sRGB values in range 0..255 to OKLab.

        Accepts arrays of shape (..., 3).
        """

        rgb = np.asarray(rgb, dtype=np.float32) / 255.0

        # sRGB gamma decoding
        rgb = np.where(
            rgb <= 0.04045,
            rgb / 12.92,
            ((rgb + 0.055) / 1.055) ** 2.4
        )

        r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]

        l = (
            0.4122214708 * r +
            0.5363325363 * g +
            0.0514459929 * b
        )

        m = (
            0.2119034982 * r +
            0.6806995451 * g +
            0.1073969566 * b
        )

        s = (
            0.0883024619 * r +
            0.2817188376 * g +
            0.6299787005 * b
        )

        l_ = np.cbrt(l)
        m_ = np.cbrt(m)
        s_ = np.cbrt(s)

        L = (
            0.2104542553 * l_ +
            0.7936177850 * m_ -
            0.0040720468 * s_
        )

        A = (
            1.9779984951 * l_ -
            2.4285922050 * m_ +
            0.4505937099 * s_
        )
```

```
        B = (
            0.0259040371 * l_ +
            0.7827717662 * m_ -
            0.8086757660 * s_
        )

        return np.stack([L, A, B], axis=-1)


    # ------------------------------------------------------------
    # NEAREST PALETTE COLOR
    # ------------------------------------------------------------

    def quantize_direct(image_lab, tree):
        """
        Find nearest palette entry for every pixel.
        Uses OKLab Euclidean distance.
        """

        pixels = image_lab.reshape(-1, 3)

        result = np.empty(len(pixels), dtype=np.uint8)

        # Process in chunks to limit memory usage
        chunk_size = 100000

        for start in range(0, len(pixels), chunk_size):
            end = min(start + chunk_size, len(pixels))

            _, indices = tree.query(
                pixels[start:end],
                workers=-1
            )

            result[start:end] = indices

        return result.reshape(image_lab.shape[:2])


    # ------------------------------------------------------------
    # FLOYD-STEINBERG DITHERING
    # ------------------------------------------------------------

    def quantize_floyd_steinberg(image_lab, tree, palette_lab):
        """
        Floyd-Steinberg error diffusion in OKLab space.
        """

        work = image_lab.copy()
        height, width, _ = work.shape

        output = np.zeros((height, width), dtype=np.uint8)

        for y in range(height):
            for x in range(width):

                old_color = work[y, x].copy()

                _, index = tree.query(old_color)

                new_color = palette_lab[index]
                output[y, x] = index
```

```
            error = old_color - new_color

            # Distribute quantization error
            if x + 1 < width:
                work[y, x + 1] += error * (7.0 / 16.0)

            if y + 1 < height:
                if x > 0:
                    work[y + 1, x - 1] += error * (3.0 / 16.0)

                work[y + 1, x] += error * (5.0 / 16.0)

                if x + 1 < width:
                    work[y + 1, x + 1] += error * (1.0 / 16.0)

    return output


# ------------------------------------------------------------
# MAIN
# ------------------------------------------------------------

def main():

    parser = argparse.ArgumentParser(
        description="Convert images to the Quake palette."
    )

    parser.add_argument("input", help="Input image")
    parser.add_argument("output", help="Output PNG")

    parser.add_argument(
        "--palette",
        default="quake.pal",
        help="JASC-PAL palette file"
    )

    parser.add_argument(
        "--dither",
        choices=["none", "floyd"],
        default="none",
        help="Dithering algorithm"
    )

    parser.add_argument(
        "--background",
        nargs=3,
        type=int,
        default=[0, 0, 0],
        metavar=("R", "G", "B"),
        help="Background color for transparent pixels"
    )

    args = parser.parse_args()

    # Load palette
    palette = load_palette(args.palette)

    print(f"Loaded {len(palette)} palette colors")

    # Convert palette to OKLab
    palette_lab = rgb_to_oklab(palette)
```

```
    # Build nearest-neighbor search tree
    tree = cKDTree(palette_lab)

    # Load image and composite transparency
    image = Image.open(args.input).convert("RGBA")

    background = Image.new(
        "RGBA",
        image.size,
        tuple(args.background) + (255,)
    )

    image = Image.alpha_composite(background, image)
    image_rgb = np.asarray(image.convert("RGB"))

    # Convert image to OKLab
    image_lab = rgb_to_oklab(image_rgb)

    print(f"Processing {image.width}x{image.height} image")

    # Quantize
    if args.dither == "floyd":
        print("Using Floyd-Steinberg dithering")

        indices = quantize_floyd_steinberg(
            image_lab,
            tree,
            palette_lab
        )

    else:
        print("Using direct OKLab quantization")

        indices = quantize_direct(
            image_lab,
            tree
        )

    # Create genuine indexed PNG
    result = Image.fromarray(indices, mode="P")

    # Set the complete palette
    result.putpalette(palette.flatten().tolist())

    # Save
    result.save(args.output, format="PNG")

    print(f"Saved: {args.output}")


if __name__ == "__main__":
    main()
```

2. Installation

Install the dependencies:

```
pip install pillow numpy scipy
```

Save your Quake palette text as:

```
quake.pal
```

The file should start with:

```
JASC-PAL
0100
256
```

Followed by your 256 RGB triplets.

3. Usage

Direct perceptual quantization:

```
python quake_quantize.py input.png output.png
```

With Floyd–Steinberg dithering:

```
python quake_quantize.py input.png output.png --dither floyd
```

Specify another palette:

```
python quake_quantize.py input.png output.png --palette quake.pal
```

Change the background used for transparent pixels:

```
python quake_quantize.py input.png output.png --background 32 32 32
```

4. What I would improve next

This implementation is the perceptual nearest-color baseline. It does not yet implement the adaptive hue redistribution we discussed.

For the next iteration, I would add three features:

Feature	Benefit
Adaptive hue redistribution	Moves poorly represented source hues toward better-supported palette regions
Lightness/chroma-dependent mapping	Prevents dark and bright colors from receiving inappropriate hue shifts
Palette-aware dithering	Reduces visible banding without introducing excessive noise

My recommendation: First compare direct OKLab quantization against Floyd–Steinberg on several different images. That establishes a baseline. Then add adaptive hue redistribution as an optional processing stage, so you can compare faithful conversion against Quake-stylized conversion using exactly the same palette and input image.

You said:

```
pip install pillow numpy scipy
error: externally-managed-environment

× This environment is externally managed
╰─> To install Python packages system-wide, try apt install
    python3-xyz, where xyz is the package you are trying to
    install.

    If you wish to install a non-Debian-packaged Python package,
    create a virtual environment using python3 -m venv path/to/venv.
```

Then use path/to/venv/bin/python and path/to/venv/bin/pip. Make sure you have python3-full installed.

If you wish to install a non-Debian packaged Python application, it may be easiest to use pipx install xyz, which will manage a virtual environment for you. Make sure you have pipx installed.

See /usr/share/doc/python3.12/README.venv for more information.

note: If you believe this is a mistake, please contact your Python installation or OS distribution provider. You can override this, at the risk of breaking your Python installation or OS, by passing --break-system-packages.
hint: See PEP 668 for the detailed specification.

ChatGPT said:

This is expected on Debian 12+ and Ubuntu systems that follow **PEP 668**. Your system Python is externally managed, so `pip` is prevented from installing packages globally.

The recommended solution is to create a virtual environment for the Quake palette script.

1. Install the virtual environment support

```
sudo apt update
sudo apt install python3-full
```

2. Create a virtual environment

From the directory containing your `quake_quantize.py` script:

```
python3 -m venv .venv
```

3. Install the dependencies inside it

```
.venv/bin/python -m pip install pillow numpy scipy
```

4. Run the script

```
.venv/bin/python quake_quantize.py input.png output.png
```

For Floyd–Steinberg dithering:

```
.venv/bin/python quake_quantize.py input.png output.png --dither floyd
```

Optional: activate the environment

Instead of typing `.venv/bin/python` every time:

```
source .venv/bin/activate
pip install pillow numpy scipy
python quake_quantize.py input.png output.png
```

When finished, leave the environment with:

```
deactivate
```

Avoid `--break-system-packages`**.** It bypasses Debian's protection and can interfere with OS-managed Python packages. A virtual environment keeps the script's dependencies separate and is the safer approach.

You said:

```
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ grep -nE 'if __name__|def main|save\(|Image.save' conv.py
192:def main():
278:    result.save(args.output, format="PNG")
283:if __name__ == "__main__":
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$
```

ChatGPT said:

That output confirms your script has:

- `main()` defined at line 192.
- The converted PNG saved at line 278.
- The Python entry point at line 283.

So `conv.py` is structured as an executable script. You can run it directly using the virtual environment:

```
.venv/bin/python conv.py input.png output.png
```

If your virtual environment is in a different directory, adjust `.venv/bin/python` accordingly.

One thing to check: your script is named `conv.py`, whereas my earlier example used `quake_quantize.py`. That's perfectly fine; just use `conv.py`.

If you get an error, paste the complete traceback and I'll help pinpoint it.

You said:

```
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ .venv/bin/python conv.py f_floortile2.tga test.png
Traceback (most recent call last):
  File "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests/conv.py", line 284, in <module>
    main()
  File "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests/conv.py", line 226, in main
    palette = load_palette(args.palette)
              ^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests/conv.py", line 38, in load_palette
    rgb = [int(v) for v in line.split()]
           ^^^^^^
ValueError: invalid literal for int() with base 10: 'this'
```

ChatGPT said:

The error is in `load_palette()`: your `.pal` file contains a non-numeric line (apparently a comment beginning with `this`), but the parser assumes every line after the header contains three RGB integers.

Your palette file likely has a comment or descriptive text that the original parser doesn't handle.

Fix `load_palette()` **in** `conv.py`

Replace the existing function (around line 30) with this more tolerant version:

```
def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = f.readlines()
```

```
    # Remove blank lines and comments
    lines = [
        line.strip()
        for line in lines
        if line.strip() and not line.strip().startswith(("//", "#", ";"))
    ]

    if len(lines) < 3:
        raise ValueError("Invalid JASC-PAL file")

    if lines[0] != "JASC-PAL":
        raise ValueError("Missing JASC-PAL header")

    if lines[1] != "0100":
        raise ValueError(f"Unsupported palette version: {lines[1]}")

    count = int(lines[2])

    colors = []
    for line_number, line in enumerate(lines[3:], start=4):
        parts = line.split()

        if len(parts) != 3:
            continue

        try:
            rgb = [int(v) for v in parts]
        except ValueError:
            continue

        if all(0 <= v <= 255 for v in rgb):
            colors.append(rgb)

    if len(colors) != count:
        raise ValueError(
            f"Palette declares {count} colors, but {len(colors)} valid RGB entries were found."
        )

    return np.array(colors, dtype=np.uint8)
```

Then try again

```
.venv/bin/python conv.py f_floortile2.tga test.png
```

Important: This version skips non-RGB lines, but still checks that exactly the declared number of colors were found. That prevents silently producing an incorrect Quake palette.

If it reports a different number of colors, show me the first 15 lines of your palette file:

```
head -15 quake.pal
```

We can then adapt the parser to its exact format.

You said:

```
venv/bin/python conv.py f_floortile2.tga test.png
  File "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests/conv.py", line 37
    with open(path, "r", encoding="utf-8-sig") as f:
    ^
IndentationError: expected an indented block after function definition on line 36
```

ChatGPT said:

The function was pasted with incorrect indentation. Python requires every line inside `def load_palette(path):` to be indented by **4 spaces**.

Replace the entire function, including its `def` line, with this exact version:

```
def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = f.readlines()

    lines = [
        line.strip()
        for line in lines
        if line.strip()
        and not line.strip().startswith(("//", "#", ";"))
    ]

    if len(lines) < 3:
        raise ValueError("Invalid JASC-PAL file")

    if lines[0] != "JASC-PAL":
        raise ValueError("Missing JASC-PAL header")

    if lines[1] != "0100":
        raise ValueError(f"Unsupported palette version: {lines[1]}")

    count = int(lines[2])
    colors = []

    for line in lines[3:]:
        parts = line.split()

        if len(parts) != 3:
            continue

        try:
            rgb = [int(v) for v in parts]
        except ValueError:
            continue

        if all(0 <= v <= 255 for v in rgb):
            colors.append(rgb)

    if len(colors) != count:
        raise ValueError(
            f"Palette declares {count} colors, "
            f"but {len(colors)} valid RGB entries were found."
        )

    return np.array(colors, dtype=np.uint8)
```

Important: Make sure `def load_palette(path):` starts at the leftmost column. Everything inside it starts four spaces to the right.

Then run:

```
.venv/bin/python conv.py f_floortile2.tga test.png
```

If indentation problems persist, you can inspect the function with:

```
sed -n '30,85p' conv.py
```

That will let me spot exactly where the indentation went wrong.

You said:

it works now! Whats the next steppie?

ChatGPT said:

Excellent! 😁 Now let's see what the Quake palette conversion actually does to your texture.

Step 1 — Inspect the result

Open `test.png` alongside the original `f_floortile2.tga`.

Pay attention to:

- Whether the original texture's overall colors are preserved.
- Whether subtle details have disappeared.
- Whether gradients have become harsh color bands.
- Whether the result looks recognizably like a Quake-era texture.

Step 2 — Try dithering

Run:

```
.venv/bin/python conv.py f_floortile2.tga test_dither.png --dither floyd
```

Compare `test.png` and `test_dither.png`.

Dithering can improve apparent tonal detail, but may introduce grain or noise into textures.

Step 3 — Check the output palette

Run:

```
.venv/bin/python -c "from PIL import Image; im=Image.open('test.png'); print('Mode:', im.mode); print('Size:', im.size); print('Colors:', len(im.getcolors(256) or []))"
```

Ideally, the output should be:

- `Mode: P` — indexed color.
- Original texture dimensions preserved.
- No more than 256 colors.

Step 4 — Show me the result

Upload **three files** if possible:

1. `f_floortile2.tga`
2. `test.png`
3. `test_dither.png`

Then we can move to the interesting part: **adaptive hue redistribution**.

That's where we'll improve the algorithm so it doesn't merely select the nearest Quake palette color, but intelligently maps source colors into the palette's available color ranges while preserving the texture's visual character.

The goal is not just fewer colors. It's making the texture look as though it was originally authored for the Quake palette.

You said:

cant upload right now but:
both png are more noisy, the dithered more than the undithered.
give me the adaptive version of the script.

ChatGPT said:

That observation is useful. It tells us two things:

1. The Quake palette has relatively poor local color coverage for some of the source texture's colors, so nearest-color quantization is introducing visible color jumps.
2. Floyd–Steinberg dithering is distributing the quantization error spatially, which makes the texture look even noisier.

For game textures, especially stone, concrete, dirt and metal, I would **not use Floyd–Steinberg as the default**. It can look good for illustrations, but often produces unwanted grain in tiled textures.

Let's change the approach.

What the adaptive version will do

The new script will:

- Analyze the source image's color distribution.
- Convert colors into OKLab perceptual space.
- Measure how well the Quake palette covers the source colors.
- Adaptively remap source hues toward better-supported palette regions.
- Preserve luminance and saturation relationships as much as possible.
- Use smooth hue adjustments rather than abrupt color substitutions.
- Keep dithering optional, disabled by default.

Importantly, it won't simply force the source image to use every hue in the Quake palette. That would produce unnatural colors. It will only redistribute hues where doing so improves palette coverage.

Adaptive Quake palette converter

This version uses a different strategy: for every distinct source color, it tests nearby hue adjustments and finds the best Quake palette match. It penalizes excessive hue changes, so it won't freely turn brown stone into blue or green just because those colors exist in the palette.

It also:

- Processes identical source colors consistently.
- Preserves lightness and chroma while testing hue changes.
- Uses OKLab perceptual distance.
- Does not apply dithering by default.
- Supports conservative and aggressive adaptation settings.

1. Replace `conv.py`

Make a backup first:

```
cp conv.py conv_old.py
```

Then replace its contents with this complete script:

```
import argparse
import numpy as np
from PIL import Image
from scipy.spatial import cKDTree
```

```
def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 3 or lines[0] != "JASC-PAL":
        raise ValueError("Invalid JASC-PAL file")

    if lines[1] != "0100":
        raise ValueError(f"Unsupported palette version: {lines[1]}")

    count = int(lines[2])
    colors = []

    for line in lines[3:]:
        parts = line.split()

        if len(parts) != 3:
            continue

        try:
            rgb = [int(v) for v in parts]
        except ValueError:
            continue

        if all(0 <= v <= 255 for v in rgb):
            colors.append(rgb)

    if len(colors) != count:
        raise ValueError(
            f"Palette declares {count} colors, "
            f"but {len(colors)} valid RGB entries were found."
        )

    return np.array(colors, dtype=np.uint8)


def srgb_to_linear(rgb):
    return np.where(
        rgb <= 0.04045,
        rgb / 12.92,
        ((rgb + 0.055) / 1.055) ** 2.4
    )


def rgb_to_oklab(rgb):
    rgb = np.asarray(rgb, dtype=np.float64) / 255.0
    rgb = srgb_to_linear(rgb)

    r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]

    l = 0.4122214708*r + 0.5363325363*g + 0.0514459929*b
    m = 0.2119034982*r + 0.6806995451*g + 0.1073969566*b
    s = 0.0883024619*r + 0.2817188376*g + 0.6299787005*b

    l_ = np.cbrt(l)
    m_ = np.cbrt(m)
    s_ = np.cbrt(s)

    L = 0.2104542553*l_ + 0.7936177850*m_ - 0.0040720468*s_
    a = 1.9779984951*l_ - 2.4285922050*m_ + 0.4505937099*s_
    b = 0.0259040371*l_ + 0.7827717662*m_ - 0.8086757660*s_
```

```
        return np.stack([L, a, b], axis=-1)


    def oklab_to_lch(lab):
        L = lab[:, 0]
        a = lab[:, 1]
        b = lab[:, 2]

        C = np.sqrt(a*a + b*b)
        H = np.arctan2(b, a)

        return L, C, H


    def lch_to_oklab(L, C, H):
        a = C * np.cos(H)
        b = C * np.sin(H)

        return np.column_stack([L, a, b])


    def adaptive_quantize(rgb, palette, strength, max_shift, hue_step):
        height, width, _ = rgb.shape

        # Work on unique source colors to avoid repeated calculations.
        unique_rgb, inverse = np.unique(
            rgb.reshape(-1, 3),
            axis=0,
            return_inverse=True
        )

        print(f"Unique source colors: {len(unique_rgb)}")

        source_lab = rgb_to_oklab(unique_rgb)
        palette_lab = rgb_to_oklab(palette)

        tree = cKDTree(palette_lab)

        L, C, H = oklab_to_lch(source_lab)

        # Candidate hue adjustments, including zero.
        shifts = np.arange(
            -max_shift,
            max_shift + hue_step * 0.5,
            hue_step
        )

        shifts = np.unique(np.append(shifts, 0.0))
        shifts_rad = np.deg2rad(shifts)

        best_score = np.full(len(unique_rgb), np.inf)
        best_index = np.zeros(len(unique_rgb), dtype=np.int32)

        for shift_deg, shift_rad in zip(shifts, shifts_rad):

            # Preserve lightness and chroma; only rotate hue.
            candidate_lab = lch_to_oklab(
                L,
                C,
                H + shift_rad
            )
```

```
            distances, indices = tree.query(candidate_lab, k=1)

            # Perceptual cost of changing the original hue.
            hue_change_cost = (
                2.0 * C * np.abs(np.sin(shift_rad / 2.0))
            )

            # Near-neutral colors should not be hue-shifted.
            hue_change_cost = np.where(
                C < 0.015,
                0.0,
                hue_change_cost
            )

            # Lower strength permits more aggressive adaptation.
            score = distances + strength * hue_change_cost

            improved = score < best_score

            best_score[improved] = score[improved]
            best_index[improved] = indices[improved]

        # Reconstruct indexed image.
        result = best_index[inverse].reshape(height, width)

        return result.astype(np.uint8)


    def main():
        parser = argparse.ArgumentParser(
            description="Adaptive Quake palette texture converter"
        )

        parser.add_argument("input")
        parser.add_argument("output")

        parser.add_argument(
            "--palette",
            default="quake.pal"
        )

        parser.add_argument(
            "--strength",
            type=float,
            default=0.65,
            help="Hue-change penalty: higher is more conservative"
        )

        parser.add_argument(
            "--max-shift",
            type=float,
            default=25.0,
            help="Maximum hue adjustment in degrees"
        )

        parser.add_argument(
            "--hue-step",
            type=float,
            default=2.0,
            help="Hue search resolution in degrees"
        )

        parser.add_argument(
```

```
        "--background",
        type=int,
        nargs=3,
        default=[0, 0, 0],
        metavar=("R", "G", "B")
    )

    args = parser.parse_args()

    if args.strength < 0:
        parser.error("--strength must not be negative")

    if args.max_shift < 0 or args.hue_step <= 0:
        parser.error("--max-shift must be nonnegative and --hue-step positive")

    print("Loading palette...")
    palette = load_palette(args.palette)

    print("Loading image...")
    image = Image.open(args.input).convert("RGBA")

    background = Image.new(
        "RGBA",
        image.size,
        tuple(args.background) + (255,)
    )

    image = Image.alpha_composite(background, image)
    rgb = np.asarray(image.convert("RGB"))

    print("Adaptive color mapping...")
    indices = adaptive_quantize(
        rgb,
        palette,
        args.strength,
        args.max_shift,
        args.hue_step
    )

    result = Image.fromarray(indices, mode="P")

    # PNG palette must contain 256 RGB entries.
    padded_palette = np.zeros((256, 3), dtype=np.uint8)
    padded_palette[:len(palette)] = palette

    result.putpalette(padded_palette.flatten().tolist())
    result.save(args.output, format="PNG")

    print(f"Saved: {args.output}")


if __name__ == "__main__":
    main()
```

2. Try these three settings

Keep `quake.pal` in the same directory as `conv.py`, unless you specify its location with `--palette`.

Recommended starting point

Conservative adaptation

```
.venv/bin/python conv.py f_floortile2.tga adaptive.png --strength 0.8
```

Moderate adaptation

```
.venv/bin/python conv.py f_floortile2.tga adaptive_medium.png --strength 0.65
```

Aggressive adaptation

```
.venv/bin/python conv.py f_floortile2.tga adaptive_strong.png --strength 0.4 --max-shift 35
```

3. What the strength parameter means

Strength	Behavior
1.0	Very conservative
0.8	Subtle correction
0.65	Balanced
0.4	Noticeable hue adaptation
0.2	Aggressive

These are starting points, not guaranteed perceptual-optimal settings.

4. What I want to investigate next

One important distinction: this version adapts each source color against the available palette. It does not yet perform global image-histogram optimization or spatial analysis.

For the next iteration, I'd like to add **texture-aware optimization**:

- Detect flat regions and preserve their color consistency.
- Detect edges and avoid introducing color noise around them.
- Analyze hue coverage across the entire image.
- Account for the Quake palette's actual color distribution.
- Optionally add subtle ordered dithering instead of error diffusion.

That should be particularly useful for game textures, where visual consistency and clean tiling matter more than photographic dithering.

You said:

ok works but still not perfect. next step please.

Worked for 8s

ChatGPT said:

The next step is to make the converter **texture-aware**, rather than simply improving its color matching.

At the moment, the adaptive script treats every unique RGB color independently. That has a weakness: two very similar colors can be mapped to different Quake palette entries, even when they belong to the same surface. This can produce speckling, inconsistent stone colors and unwanted visual variation.

I'd tackle this in three stages:

1. **Edge-aware quantization:** Keep neighboring pixels visually consistent in flat regions, but preserve genuine texture edges.
2. **Palette-aware color clustering:** Group similar source colors before mapping them, so surfaces retain coherent color relationships.
3. **Optional ordered dithering:** Add controlled tonal detail without the grain produced by Floyd–Steinberg.

Before I write the next version, I need to know what is bothering you most about the current result.

What is the main remaining problem?

- **A:** Colors are too different from the original.
- **B:** The texture looks noisy or speckled.
- **C:** Too much detail and subtle shading are lost.
- **D:** Colors look washed out or too saturated.
- **E:** It doesn't look sufficiently like an authentic Quake-era texture.

You can pick more than one. That will determine which part of the algorithm we improve first.

You said:

B is the main problem followed by E. Id dont care if the color saturation or hue is slightly off compared to the original. They will never be seen together.

Worked for 37s

ChatGPT said:

Perfect. That changes the optimization target considerably. We should stop prioritizing faithful reproduction of the source colors and prioritize **coherent, deliberately limited Quake-style textures**.

For your purposes, the ideal result should have:

- Broad, consistent color regions instead of isolated colored pixels.
- Clean transitions between different materials and surface details.
- Stronger use of the Quake palette's characteristic color ramps.
- No mandatory dithering.
- Some freedom to shift hues and saturation to get a convincing retro appearance.

Next version: edge-aware adaptive quantization

I'm going to add a bilateral filter in OKLab space before palette mapping.

Unlike ordinary blur, a bilateral filter considers both spatial distance and color difference. It smooths small color variations within a surface while trying to preserve actual edges.

The pipeline becomes:

1. Analyze the source image.
2. Smooth small color variations without indiscriminately blurring edges.
3. Adapt colors toward available Quake palette colors.
4. Quantize without dithering.
5. Save as an indexed 8-bit PNG.

This is aimed directly at problem B, while the more permissive color adaptation helps with E.

1. Replace `conv.py` with this

```
import argparse
import numpy as np
from PIL import Image
from scipy.spatial import cKDTree


def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = [
            line.strip() for line in f
            if line.strip()
        ]
```

```
        if len(lines) < 3 or lines[0] != "JASC-PAL":
            raise ValueError("Invalid JASC-PAL file")

        if lines[1] != "0100":
            raise ValueError(f"Unsupported palette version: {lines[1]}")

        count = int(lines[2])
        colors = []

        for line in lines[3:]:
            parts = line.split()

            if len(parts) != 3:
                continue

            try:
                rgb = [int(v) for v in parts]
            except ValueError:
                continue

            if all(0 <= v <= 255 for v in rgb):
                colors.append(rgb)

        if len(colors) != count:
            raise ValueError(
                f"Palette declares {count} colors, "
                f"but {len(colors)} valid RGB entries were found."
            )

        return np.array(colors, dtype=np.uint8)


    def srgb_to_linear(rgb):
        return np.where(
            rgb <= 0.04045,
            rgb / 12.92,
            ((rgb + 0.055) / 1.055) ** 2.4
        )


    def linear_to_srgb(rgb):
        return np.where(
            rgb <= 0.0031308,
            rgb * 12.92,
            1.055 * np.maximum(rgb, 0) ** (1 / 2.4) - 0.055
        )


    def rgb_to_oklab(rgb):
        rgb = np.asarray(rgb, dtype=np.float64) / 255.0
        rgb = srgb_to_linear(rgb)

        r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]

        l = 0.4122214708*r + 0.5363325363*g + 0.0514459929*b
        m = 0.2119034982*r + 0.6806995451*g + 0.1073969566*b
        s = 0.0883024619*r + 0.2817188376*g + 0.6299787005*b

        l = np.cbrt(l)
        m = np.cbrt(m)
        s = np.cbrt(s)
```

```
        L = 0.2104542553*l + 0.7936177850*m - 0.0040720468*s
        a = 1.9779984951*l - 2.4285922050*m + 0.4505937099*s
        b = 0.0259040371*l + 0.7827717662*m - 0.8086757660*s

        return np.stack([L, a, b], axis=-1)


    def oklab_to_rgb(lab):
        L, a, b = lab[..., 0], lab[..., 1], lab[..., 2]

        l = (L + 0.3963377774*a + 0.2158037573*b) ** 3
        m = (L - 0.1055613458*a - 0.0638541728*b) ** 3
        s = (L - 0.0894841775*a - 1.2914855480*b) ** 3

        r = 4.0767416621*l - 3.3077115913*m + 0.2309699292*s
        g = -1.2684380046*l + 2.6097574011*m - 0.3413193965*s
        b = -0.0041960863*l - 0.7034186147*m + 1.7076147010*s

        rgb = np.stack([r, g, b], axis=-1)
        rgb = linear_to_srgb(rgb)

        return np.clip(rgb * 255.0, 0, 255).astype(np.uint8)


    def bilateral_smooth(lab, radius=2, spatial_sigma=1.2,
                         range_sigma=0.045):

        if radius <= 0 or range_sigma <= 0:
            return lab

        height, width = lab.shape[:2]

        padded = np.pad(
            lab,
            ((radius, radius), (radius, radius), (0, 0)),
            mode="reflect"
        )

        center = padded[
            radius:radius + height,
            radius:radius + width
        ]

        weighted_sum = np.zeros_like(center)
        weight_sum = np.zeros((height, width), dtype=np.float64)

        spatial_denominator = 2.0 * spatial_sigma**2
        range_denominator = 2.0 * range_sigma**2

        for dy in range(-radius, radius + 1):
            for dx in range(-radius, radius + 1):

                neighbor = padded[
                    radius + dy:radius + dy + height,
                    radius + dx:radius + dx + width
                ]

                color_diff = np.sum(
                    (neighbor - center) ** 2,
                    axis=2
                )

                spatial_dist = dx*dx + dy*dy
```

```
            weight = np.exp(
                -spatial_dist / spatial_denominator
                -color_diff / range_denominator
            )

            weighted_sum += neighbor * weight[..., None]
            weight_sum += weight

    return weighted_sum / weight_sum[..., None]


def lch_to_oklab(L, C, H):
    return np.column_stack([
        L,
        C * np.cos(H),
        C * np.sin(H)
    ])


def adaptive_quantize(rgb, palette, strength, max_shift,
                      hue_step, smooth, radius):

    height, width = rgb.shape[:2]

    print("Converting image to OKLab...")
    lab = rgb_to_oklab(rgb)

    if smooth > 0:
        print("Applying edge-aware smoothing...")
        lab = bilateral_smooth(
            lab,
            radius=radius,
            range_sigma=smooth
        )

    print("Preparing palette...")
    palette_lab = rgb_to_oklab(palette)
    tree = cKDTree(palette_lab)

    # Reduce repeated calculations while retaining subtle color differences.
    unique_lab, inverse = np.unique(
        np.round(lab.reshape(-1, 3), decimals=4),
        axis=0,
        return_inverse=True
    )

    print(f"Colors to process: {len(unique_lab)}")

    L = unique_lab[:, 0]
    a = unique_lab[:, 1]
    b = unique_lab[:, 2]

    C = np.sqrt(a*a + b*b)
    H = np.arctan2(b, a)

    shifts = np.arange(
        -max_shift,
        max_shift + hue_step * 0.5,
        hue_step
    )

    shifts = np.unique(np.append(shifts, 0.0))
```

```
        best_score = np.full(len(unique_lab), np.inf)
        best_index = np.zeros(len(unique_lab), dtype=np.int32)

        print("Searching adaptive palette matches...")

        for shift in shifts:

            shift_rad = np.deg2rad(shift)

            candidate_lab = lch_to_oklab(
                L,
                C,
                H + shift_rad
            )

            distances, indices = tree.query(candidate_lab, k=1)

            # Penalize excessive hue changes.
            hue_cost = (
                2.0 * C * abs(np.sin(shift_rad / 2.0))
            )

            # Hue is unreliable for almost-neutral colors.
            hue_cost = np.where(C < 0.015, 0.0, hue_cost)

            score = distances + strength * hue_cost

            improved = score < best_score

            best_score[improved] = score[improved]
            best_index[improved] = indices[improved]

        result = best_index[inverse].reshape(height, width)

        return result.astype(np.uint8)


    def main():
        parser = argparse.ArgumentParser(
            description="Texture-aware adaptive Quake palette converter"
        )

        parser.add_argument("input")
        parser.add_argument("output")

        parser.add_argument("--palette", default="quake.pal")

        parser.add_argument(
            "--strength",
            type=float,
            default=0.4,
            help="Hue-change penalty; lower means more aggressive adaptation"
        )

        parser.add_argument(
            "--max-shift",
            type=float,
            default=35.0,
            help="Maximum hue adjustment in degrees"
        )

        parser.add_argument(
```

```
    "--hue-step",
    type=float,
    default=2.0
)

parser.add_argument(
    "--smooth",
    type=float,
    default=0.045,
    help="Bilateral smoothing strength; 0 disables smoothing"
)

parser.add_argument(
    "--radius",
    type=int,
    default=2,
    help="Bilateral filter radius in pixels"
)

parser.add_argument(
    "--background",
    type=int,
    nargs=3,
    default=[0, 0, 0],
    metavar=("R", "G", "B")
)

args = parser.parse_args()

if args.strength < 0:
    parser.error("--strength must not be negative")

if args.max_shift < 0 or args.hue_step <= 0:
    parser.error("Invalid hue settings")

if args.smooth < 0 or args.radius < 0:
    parser.error("Invalid smoothing settings")

print("Loading palette...")
palette = load_palette(args.palette)

print("Loading texture...")
image = Image.open(args.input).convert("RGBA")

background = Image.new(
    "RGBA",
    image.size,
    tuple(args.background) + (255,)
)

image = Image.alpha_composite(background, image)
rgb = np.asarray(image.convert("RGB"))

indices = adaptive_quantize(
    rgb,
    palette,
    args.strength,
    args.max_shift,
    args.hue_step,
    args.smooth,
    args.radius
)
```

```
    result = Image.fromarray(indices, mode="P")

    padded_palette = np.zeros((256, 3), dtype=np.uint8)
    padded_palette[:len(palette)] = palette

    result.putpalette(padded_palette.flatten().tolist())
    result.save(args.output, format="PNG")

    print(f"Saved: {args.output}")


if __name__ == "__main__":
    main()
```

2. First test: three smoothing levels

Use the same input texture for all three tests.

A — Light smoothing

```
.venv/bin/python conv.py f_floortile2.tga test_light.png --smooth 0.025
```

B — Medium smoothing (default)

```
.venv/bin/python conv.py f_floortile2.tga test_medium.png --smooth 0.045
```

C — Strong smoothing

```
.venv/bin/python conv.py f_floortile2.tga test_strong.png --smooth 0.075 --radius 3
```

How to interpret the results

Result	What it tells us
Light is still speckled	Increase smoothing slightly
Medium looks clean	Good starting point
Strong loses surface detail	Reduce smoothing
All three look noisy	The palette mapping itself needs further work

3. My next planned improvement: Quake-style color ramps

There is another important opportunity here.

Rather than simply finding the nearest palette entry, we can identify groups of colors in the Quake palette that form coherent ramps:

- Dark brown → medium brown → light brown.
- Dark grey → medium grey → light grey.
- Dark green → medium green → light green.
- Dark red → medium red → light red.

Then we can map entire surface regions to these ramps instead of independently selecting arbitrary colors.

That is likely to make a much bigger difference to the authentic Quake appearance than increasingly aggressive hue manipulation.

For now, compare the three smoothing results. The important question is whether the speckling disappears without making the floor texture look like a blurry painting.

You said:

i guess light is the best. even medium loses notable detail. Lets go for the ramps now.

ChatGPT said:

Good. That tells us we should preserve fine surface detail and solve the noise problem through **palette selection**, not stronger image smoothing.

For the next version, we'll introduce palette ramps.

The key change is that we won't treat all 256 Quake colors as an unordered collection. We'll organize them into related color families and use those families as coherent shading ramps.

For example, a brown stone surface should ideally use a sequence of related dark, medium and light browns rather than jumping between unrelated brown, purple and grey entries.

We'll use:

- Very light bilateral smoothing (`0.025`).
- Automatic palette ramp discovery.
- Per-pixel matching constrained to selected ramps.
- Adaptive selection of ramps based on the image's overall color distribution.
- No dithering.

One important detail: we'll discover ramps from the actual palette rather than hardcoding assumed Quake color ranges. That makes the script work with your supplied JASC-PAL file.

Version 4: Quake palette ramp mapping

This version discovers hue families in your actual palette, separates neutral colours, and sorts each family by lightness. Each source colour is assigned to a family first; it then selects a shade from that family's ramp.

This is different from the previous algorithm: it deliberately sacrifices some source-colour accuracy to encourage consistent shading.

1. Replace `conv.py`

Keep your existing backup. Replace the script with this:

```
import argparse
import numpy as np
from PIL import Image


# ------------------------------------------------------------
# Palette loading
# ------------------------------------------------------------

def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 3 or lines[0] != "JASC-PAL":
        raise ValueError("Invalid JASC-PAL file")

    if lines[1] != "0100":
        raise ValueError(f"Unsupported palette version: {lines[1]}")

    count = int(lines[2])
    colors = []

    for line in lines[3:]:
```

```
            parts = line.split()

            if len(parts) != 3:
                continue

            try:
                rgb = [int(v) for v in parts]
            except ValueError:
                continue

            if all(0 <= v <= 255 for v in rgb):
                colors.append(rgb)

        if len(colors) != count:
            raise ValueError(
                f"Expected {count} colors, found {len(colors)}"
            )

        return np.array(colors, dtype=np.uint8)


    # ------------------------------------------------------------
    # RGB -> OKLab
    # ------------------------------------------------------------

    def srgb_to_linear(rgb):
        return np.where(
            rgb <= 0.04045,
            rgb / 12.92,
            ((rgb + 0.055) / 1.055) ** 2.4
        )


    def rgb_to_oklab(rgb):
        rgb = np.asarray(rgb, dtype=np.float64) / 255.0
        rgb = srgb_to_linear(rgb)

        r, g, b = rgb[..., 0], rgb[..., 1], rgb[..., 2]

        l = 0.4122214708*r + 0.5363325363*g + 0.0514459929*b
        m = 0.2119034982*r + 0.6806995451*g + 0.1073969566*b
        s = 0.0883024619*r + 0.2817188376*g + 0.6299787005*b

        l = np.cbrt(l)
        m = np.cbrt(m)
        s = np.cbrt(s)

        L = 0.2104542553*l + 0.7936177850*m - 0.0040720468*s
        a = 1.9779984951*l - 2.4285922050*m + 0.4505937099*s
        b = 0.0259040371*l + 0.7827717662*m - 0.8086757660*s

        return np.stack([L, a, b], axis=-1)


    def lab_components(lab):
        L = lab[..., 0]
        a = lab[..., 1]
        b = lab[..., 2]

        C = np.sqrt(a*a + b*b)
        H = np.arctan2(b, a)

        return L, C, H
```

```
# ------------------------------------------------------------
# Edge-aware smoothing
# ------------------------------------------------------------

def bilateral_smooth(lab, radius=2, spatial_sigma=1.2,
                     range_sigma=0.025):

    if radius <= 0 or range_sigma <= 0:
        return lab

    height, width = lab.shape[:2]

    padded = np.pad(
        lab,
        ((radius, radius), (radius, radius), (0, 0)),
        mode="reflect"
    )

    center = padded[
        radius:radius + height,
        radius:radius + width
    ]

    weighted_sum = np.zeros_like(center)
    weight_sum = np.zeros((height, width), dtype=np.float64)

    for dy in range(-radius, radius + 1):
        for dx in range(-radius, radius + 1):

            neighbor = padded[
                radius + dy:radius + dy + height,
                radius + dx:radius + dx + width
            ]

            color_diff = np.sum(
                (neighbor - center)**2,
                axis=2
            )

            spatial_weight = np.exp(
                -(dx*dx + dy*dy) / (2 * spatial_sigma**2)
            )

            color_weight = np.exp(
                -color_diff / (2 * range_sigma**2)
            )

            weight = spatial_weight * color_weight

            weighted_sum += neighbor * weight[..., None]
            weight_sum += weight

    return weighted_sum / weight_sum[..., None]


# ------------------------------------------------------------
# Discover palette ramps
# ------------------------------------------------------------

def angle_difference(a, b):
    return np.abs(np.arctan2(
```

```
            np.sin(a - b),
            np.cos(a - b)
        ))


    def discover_ramps(palette_lab, neutral_threshold=0.025,
                       sectors=12):

        L, C, H = lab_components(palette_lab)

        neutral = np.where(C < neutral_threshold)[0]
        chromatic = np.where(C >= neutral_threshold)[0]

        groups = []

        # Neutral colors form their own lightness ramp.
        if len(neutral) >= 3:
            groups.append({
                "name": "Neutral",
                "indices": neutral.tolist()
            })

        # Divide chromatic palette colors into hue sectors.
        sector_groups = [[] for _ in range(sectors)]

        for index in chromatic:
            sector = int(
                np.floor((H[index] + np.pi) /
                         (2 * np.pi) * sectors)
            ) % sectors

            sector_groups[sector].append(int(index))

        # Keep sufficiently populated sectors as ramps.
        valid_groups = [
            group for group in sector_groups
            if len(group) >= 3
        ]

        if not valid_groups:
            return [{
                "name": "Full palette",
                "indices": list(range(len(palette_lab)))
            }]

        # Merge sparse sectors into their closest valid hue sector.
        valid_centers = []

        for group in valid_groups:
            angles = H[group]
            center = np.arctan2(
                np.sin(angles).mean(),
                np.cos(angles).mean()
            )
            valid_centers.append(center)

        for group in sector_groups:
            if len(group) >= 3:
                continue

            if not group:
                continue
```

```
            for index in group:
                closest = np.argmin([
                    angle_difference(H[index], center)
                    for center in valid_centers
                ])
                valid_groups[closest].append(index)

        # Sort each family from dark to light.
        for i, group in enumerate(valid_groups):
            group.sort(key=lambda index: L[index])

            angles = H[group]
            center = np.arctan2(
                np.sin(angles).mean(),
                np.cos(angles).mean()
            )

            center_degrees = int(
                np.degrees(center) % 360
            )

            groups.append({
                "name": f"Hue {center_degrees} deg",
                "indices": group
            })

        return groups


    # ------------------------------------------------------------
    # Ramp-based quantization
    # ------------------------------------------------------------

    def quantize_with_ramps(source_lab, palette_lab, ramps,
                            hue_weight=0.5, chroma_weight=0.5):

        L, C, H = lab_components(source_lab)

        pL, pC, pH = lab_components(palette_lab)

        count = len(source_lab)

        best_index = np.zeros(count, dtype=np.int32)
        best_score = np.full(count, np.inf)

        print("\nDiscovered palette ramps:")

        ramp_data = []

        for ramp in ramps:
            indices = np.array(ramp["indices"], dtype=np.int32)

            print(
                f"  {ramp['name']}: {len(indices)} colors"
            )

            # Average hue and chroma define the family.
            angles = pH[indices]

            center_hue = np.arctan2(
                np.sin(angles).mean(),
                np.cos(angles).mean()
            )
```

```
            center_chroma = np.median(pC[indices])

            ramp_data.append((
                ramp,
                indices,
                center_hue,
                center_chroma
            ))

        print("\nMapping source colors into ramps...")

        for ramp, indices, center_hue, center_chroma in ramp_data:

            # Determine whether this ramp is appropriate
            # for each source color.
            hue_diff = angle_difference(H, center_hue)

            # Hue matters less for weakly saturated colors.
            hue_cost = (
                hue_weight *
                hue_diff *
                np.minimum(C, 0.2)
            )

            chroma_cost = (
                chroma_weight *
                np.abs(C - center_chroma)
            )

            family_cost = hue_cost + chroma_cost

            # Find the best color within this ramp.
            #
            # Lightness is the dominant component:
            # this encourages coherent tonal shading.
            local_scores = (
                (L[:, None] - pL[indices][None, :])**2
                + 0.45 * (
                    C[:, None] - pC[indices][None, :]
                )**2
                + 0.10 * (
                    np.minimum(C[:, None], 0.2)
                    * angle_difference(
                        H[:, None],
                        pH[indices][None, :]
                    )
                )**2
            )

            local_positions = np.argmin(local_scores, axis=1)

            local_indices = indices[local_positions]

            local_scores = local_scores[
                np.arange(count),
                local_positions
            ]

            # Family selection plus shade selection.
            score = local_scores + family_cost**2

            improved = score < best_score
```

```
            best_score[improved] = score[improved]
            best_index[improved] = local_indices[improved]

        return best_index.astype(np.uint8)


    # ------------------------------------------------------------
    # Main
    # ------------------------------------------------------------

    def main():

        parser = argparse.ArgumentParser(
            description="Quake palette ramp texture converter"
        )

        parser.add_argument("input")
        parser.add_argument("output")

        parser.add_argument("--palette", default="quake.pal")

        parser.add_argument(
            "--smooth",
            type=float,
            default=0.025
        )

        parser.add_argument(
            "--radius",
            type=int,
            default=2
        )

        parser.add_argument(
            "--neutral-threshold",
            type=float,
            default=0.025
        )

        parser.add_argument(
            "--hue-weight",
            type=float,
            default=0.5
        )

        parser.add_argument(
            "--chroma-weight",
            type=float,
            default=0.5
        )

        parser.add_argument(
            "--background",
            type=int,
            nargs=3,
            default=[0, 0, 0]
        )

        args = parser.parse_args()

        if args.smooth < 0 or args.radius < 0:
            parser.error("Invalid smoothing settings")
```

```
print("Loading palette...")
palette = load_palette(args.palette)

palette_lab = rgb_to_oklab(palette)

print("Discovering palette ramps...")
ramps = discover_ramps(
    palette_lab,
    neutral_threshold=args.neutral_threshold
)

print("Loading texture...")
image = Image.open(args.input).convert("RGBA")

background = Image.new(
    "RGBA",
    image.size,
    tuple(args.background) + (255,)
)

image = Image.alpha_composite(background, image)

rgb = np.asarray(image.convert("RGB"))

print("Converting source to OKLab...")
lab = rgb_to_oklab(rgb)

if args.smooth > 0:
    print("Applying light bilateral smoothing...")
    lab = bilateral_smooth(
        lab,
        radius=args.radius,
        range_sigma=args.smooth
    )

# Unique colors reduce the amount of mapping work.
unique_lab, inverse = np.unique(
    np.round(lab.reshape(-1, 3), 4),
    axis=0,
    return_inverse=True
)

print(f"Unique colors: {len(unique_lab)}")

indices = quantize_with_ramps(
    unique_lab,
    palette_lab,
    ramps,
    args.hue_weight,
    args.chroma_weight
)

result_indices = indices[inverse].reshape(
    image.height,
    image.width
)

result = Image.fromarray(
    result_indices.astype(np.uint8),
    mode="P"
)
```

```
    padded_palette = np.zeros((256, 3), dtype=np.uint8)
    padded_palette[:len(palette)] = palette

    result.putpalette(padded_palette.flatten().tolist())

    result.save(args.output, format="PNG")

    print(f"\nSaved: {args.output}")


if __name__ == "__main__":
    main()
```

2. Generate three test images

Start with the defaults:

```
.venv/bin/python conv.py f_floortile2.tga ramps_default.png
```

Then try stronger hue-family selection:

```
.venv/bin/python conv.py f_floortile2.tga ramps_strong.png --hue-weight 1.0 --chroma-weight 0.8
```

And try more lightness-oriented shading:

```
.venv/bin/python conv.py f_floortile2.tga ramps_shading.png --hue-weight 0.25 --chroma-weight 0.25
```

3. What I want to evaluate

The three important things are:

1. Does the speckling decrease?
2. Do large areas of stone now have coherent shades?
3. Does the result look more like a texture authored for Quake rather than a modern texture run through a color reducer?

One caveat: These are automatically discovered hue families, not yet manually curated Quake material ramps. Some families may contain unrelated palette colors. The printed ramp counts will help us see how the palette is being divided.

4. The next major step

Once we have these results, I'd like to build a palette-ramp inspector that outputs a contact sheet of every discovered ramp, with its colors arranged from dark to light.

That lets us identify which ramps actually make sense for stone, metal, wood, dirt and other materials. We can then build a much more convincing material-aware Quake texture converter instead of relying entirely on automatic hue grouping.

You said:

> it just looks as if it uses fewer colors tbh. Especiallay bluish areas are way less detailed and could just more shifted to greeen. Also some earthy stuff seems to be not shure if oriangy or greenish, Sounds good. Also remember to leave out the fullbrights

Worked for 11s

ChatGPT said:

Agreed. I think we've found the weakness in the automatic ramp approach: it divides the palette into mathematical hue groups, not meaningful material-color families.

For example:

- Blue and cyan are relatively poorly represented, so we should allow those source colors to migrate toward green.
- Earthy colors need a deliberate brown/olive/ochre family, rather than being arbitrarily assigned to green or orange.
- The ramp selection must preserve more of the palette's available tonal detail.

And yes: **fullbright colors must be excluded entirely from normal texture conversion.**

For the conventional Quake palette, indices **224–255** are the fullbright range. We'll reserve those indices and exclude them from all ramp discovery and normal color matching.

Step 1 — Build a palette ramp inspector

Before changing the conversion algorithm again, let's see what the palette actually contains.

Create a new file called `inspect_palette.py`:

```
import argparse
import math
import colorsys
import numpy as np
from PIL import Image, ImageDraw, ImageFont


def load_palette(path):
    with open(path, "r", encoding="utf-8-sig") as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 3 or lines[0] != "JASC-PAL":
        raise ValueError("Invalid JASC-PAL file")

    count = int(lines[2])
    colors = []

    for line in lines[3:]:
        parts = line.split()
        if len(parts) != 3:
            continue

        try:
            rgb = [int(v) for v in parts]
        except ValueError:
            continue

        if all(0 <= v <= 255 for v in rgb):
            colors.append(rgb)

    if len(colors) != count:
        raise ValueError(f"Expected {count} colors, found {len(colors)}")

    return np.array(colors, dtype=np.uint8)


def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--palette", default="quake.pal")
    parser.add_argument("--fullbright-start", type=int, default=224)
    parser.add_argument("--columns", type=int, default=16)
    args = parser.parse_args()

    palette = load_palette(args.palette)

    # Exclude fullbrights.
```

```
usable = palette[:args.fullbright_start]

entries = []

for index, rgb in enumerate(usable):
    r, g, b = rgb.astype(float) / 255.0
    h, s, v = colorsys.rgb_to_hsv(r, g, b)

    entries.append({
        "index": index,
        "rgb": tuple(int(x) for x in rgb),
        "hue": h,
        "sat": s,
        "value": v
    })

# Sort by hue, then brightness.
entries.sort(key=lambda e: (
    e["hue"],
    e["value"]
))

cell_w = 112
cell_h = 58
columns = args.columns
rows = math.ceil(len(entries) / columns)

margin = 12
width = margin * 2 + columns * cell_w
height = margin * 2 + rows * cell_h

image = Image.new("RGB", (width, height), (35, 35, 35))
draw = ImageDraw.Draw(image)

try:
    font = ImageFont.truetype("DejaVuSans.ttf", 10)
except OSError:
    font = ImageFont.load_default()

for i, entry in enumerate(entries):
    x = margin + (i % columns) * cell_w
    y = margin + (i // columns) * cell_h

    color = entry["rgb"]

    draw.rectangle(
        [x, y, x + cell_w - 5, y + 32],
        fill=color
    )

    draw.text(
        (x, y + 35),
        f'{entry["index"]:03d}  {color}',
        fill=(240, 240, 240),
        font=font
    )

image.save("palette_inspector.png")

print(f"Usable colors: {len(usable)}")
print(f"Excluded fullbrights: {len(palette) - len(usable)}")
print("Saved palette_inspector.png")
```

```
if __name__ == "__main__":
    main()
```

Run:

```
.venv/bin/python inspect_palette.py
```

Open `palette_inspector.png`.

You'll see all 224 normal palette colors, sorted by hue and brightness, with their original palette indices.

You can also inspect the fullbright range separately later if needed, but it won't enter our normal conversion.

Step 2 — What we're looking for

In particular, I want to identify these families:

Green / olive

Moss, aged stone, military surfaces and greenish shadows.

Earth / ochre

Brown, orange, tan and olive-brown shades. These need careful grouping.

Blue / teal

Rather than forcing blue into a sparse blue ramp, we should allow blue-to-teal-to-green mapping.

Neutral / grey

A separate ramp for concrete, metal and stone.

Step 3 — The next converter architecture

I want to move away from automatic hue sectors and use manually curated, overlapping color families.

For example:

Family	Allowed colors
Earth	Brown, ochre, orange-brown
Olive	Yellow-green, olive, dark green
Green	Green, teal-green
Blue-green	Blue, cyan, teal, green
Neutral	Grey, near-black, near-white

The families should overlap deliberately. A blue source pixel should be allowed to select a greenish palette ramp if that gives a better Quake-style result.

We will also select ramps based on the entire texture rather than deciding independently for every pixel. That should prevent the uncertain orange-versus-green assignment you noticed.

Your next task: Run `inspect_palette.py` and, if possible, upload `palette_inspector.png` when uploads are available. We can use the actual palette entries to construct the material families instead of guessing what the palette contains.

One technical note: the fullbright cutoff is configurable with `--fullbright-start`. I'm assuming the standard final-32-index convention for your palette; if your Soldier of Fortune palette uses a different convention, we'll adjust it.

You said:

Cant upload but the palette_inspector.png is here:
http://www.flsne.de/loesch/palette_inspector.png

Worked for 9s

ChatGPT said:

I tried opening the image at both the HTTP and HTTPS addresses, but the image server returned a **502 Bad Gateway** to my retrieval service. I can't inspect the actual colors yet.

There is, however, something important I want to change before we build the next converter.

I don't want to divide the palette into arbitrary hue sectors anymore. The palette's original index order may contain useful shading relationships that the hue-sorted inspector hides.

One quick additional diagnostic

Run this from your `tests` directory:

```
.venv/bin/python -c '
from conv import load_palette
import colorsys

p = load_palette("quake.pal")

for i, rgb in enumerate(p):
    if i >= 224:
        continue
    r, g, b = rgb / 255.0
    h, s, v = colorsys.rgb_to_hsv(r, g, b)
    print(f"{i:3d}: RGB {tuple(rgb):15} H={h*360:6.1f} S={s:.2f} V={v:.2f}")
' > palette_data.txt
```

This creates `palette_data.txt` containing all 224 normal palette entries, with their original indices and HSV values.

What I want to do with that information

The next converter should have:

1. **Curated overlapping ramps:** Earth, ochre, olive, green, teal, blue-green and neutral.
2. **Blue-to-green adaptation:** Allow underrepresented blues to migrate toward teal and green, retaining tonal detail.
3. **Earth-tone resolution:** Treat brown, orange-brown and olive as related but distinct material families.
4. **Fullbright exclusion:** Never select palette indices 224–255.
5. **Consistent surface shading:** Choose a suitable family for a texture or region, then preserve its light-to-dark relationships.

Please paste the contents of `palette_data.txt` here. It will let me construct the ramps from your actual palette rather than guessing. We can then build a substantially better converter.

You said:

```
.venv/bin/python gentext.py
  File "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests/gentext.py", line 1
    .venv/bin/python -c '
                     ^
SyntaxError: unterminated string literal (detected at line 1)
```

ChatGPT said:

The command was accidentally saved **inside** `gentext.py`, so Python is trying to interpret the shell command as Python code. That's why you get the unterminated string error.

You don't need a Python script for this. Run the following directly in your terminal from the `tests` directory:

```
.venv/bin/python -c '
from conv import load_palette
import colorsys

p = load_palette("quake.pal")

for i, rgb in enumerate(p):
    if i >= 224:
        continue
    r, g, b = (int(x) / 255.0 for x in rgb)
    h, s, v = colorsys.rgb_to_hsv(r, g, b)
    print(f"{i:3d}: RGB {tuple(int(x) for x in rgb)} H={h*360:6.1f} S={s:.2f} V={v:.2f}")
' > palette_data.txt
```

Then check that the file was created:

```
head -20 palette_data.txt
```

Paste the contents of `palette_data.txt` here. It will give me the original palette indices and color values, excluding indices 224–255, so I can identify the actual palette ramps rather than guessing at hue groups.

If you want to keep `gentext.py`, delete its contents; it isn't needed for this step.

You said:

```
:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ .venv/bin/python -c '
from conv import load_palette
import colorsys

p = load_palette("quake.pal")

for i, rgb in enumerate(p):
    if i >= 224:
        continue
    r, g, b = (int(x) / 255.0 for x in rgb)
    h, s, v = colorsys.rgb_to_hsv(r, g, b)
    print(f"{i:3d}: RGB {tuple(int(x) for x in rgb)} H={h*360:6.1f} S={s:.2f} V={v:.2f}")
' > palette_data.txt
```

ChatGPT said:

That command is correct. If it returned to the terminal prompt without an error, it should have created `palette_data.txt` in the current directory.

Now run:

```
head -30 palette_data.txt
```

To check the file's size and line count:

```
wc -l palette_data.txt
```

Then paste the output here. If `head` prints nothing, check whether the file is empty with:

```
ls -l palette_data.txt
```

If there's an error, paste the exact error message instead.

You said:

```
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ head -30 palette_data.txt
  0: RGB (0, 0, 0) H=   0.0 S=0.00 V=0.00
  1: RGB (15, 15, 15) H=   0.0 S=0.00 V=0.06
  2: RGB (31, 31, 31) H=   0.0 S=0.00 V=0.12
  3: RGB (47, 47, 47) H=   0.0 S=0.00 V=0.18
  4: RGB (63, 63, 63) H=   0.0 S=0.00 V=0.25
  5: RGB (75, 75, 75) H=   0.0 S=0.00 V=0.29
  6: RGB (91, 91, 91) H=   0.0 S=0.00 V=0.36
  7: RGB (107, 107, 107) H=   0.0 S=0.00 V=0.42
  8: RGB (123, 123, 123) H=   0.0 S=0.00 V=0.48
  9: RGB (139, 139, 139) H=   0.0 S=0.00 V=0.55
 10: RGB (155, 155, 155) H=   0.0 S=0.00 V=0.61
 11: RGB (171, 171, 171) H=   0.0 S=0.00 V=0.67
 12: RGB (187, 187, 187) H=   0.0 S=0.00 V=0.73
 13: RGB (203, 203, 203) H=   0.0 S=0.00 V=0.80
 14: RGB (219, 219, 219) H=   0.0 S=0.00 V=0.86
 15: RGB (235, 235, 235) H=   0.0 S=0.00 V=0.92
 16: RGB (15, 11, 7) H=  30.0 S=0.53 V=0.06
 17: RGB (23, 15, 11) H=  20.0 S=0.52 V=0.09
 18: RGB (31, 23, 11) H=  36.0 S=0.65 V=0.12
 19: RGB (39, 27, 15) H=  30.0 S=0.62 V=0.15
 20: RGB (47, 35, 19) H=  34.3 S=0.60 V=0.18
 21: RGB (55, 43, 23) H=  37.5 S=0.58 V=0.22
 22: RGB (63, 47, 23) H=  36.0 S=0.63 V=0.25
 23: RGB (75, 55, 27) H=  35.0 S=0.64 V=0.29
 24: RGB (83, 59, 27) H=  34.3 S=0.67 V=0.33
 25: RGB (91, 67, 31) H=  36.0 S=0.66 V=0.36
 26: RGB (99, 75, 31) H=  38.8 S=0.69 V=0.39
 27: RGB (107, 83, 31) H=  41.1 S=0.71 V=0.42
 28: RGB (115, 87, 31) H=  40.0 S=0.73 V=0.45
 29: RGB (123, 95, 35) H=  40.9 S=0.72 V=0.48
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ wc -l palette_data.txt
224 palette_data.txt
```

ChatGPT said:

Excellent. The file contains all **224 non-fullbright entries**, and the first 30 already reveal something useful: this palette is arranged in deliberate, contiguous color ramps—not randomly ordered colors.

For example, indices 16–29 form a brown/ochre ramp that gradually increases in brightness and shifts slightly in hue. That is exactly the structure we want the converter to exploit. Instead of choosing unrelated nearest colors pixel by pixel, we can map image regions to coherent ramps and retain smooth shading.

Please send the remaining entries so I can map the complete palette structure:

```
sed -n '31,224p' palette_data.txt
```

That prints palette indices 30–223. You can paste the output directly here, even if it takes several messages.

I'll use it to identify the actual brown, olive, green, blue-green, neutral and other ramps, then build the converter around those ramps. Fullbrights will remain excluded from color selection, and I'll keep the light smoothing level you preferred (`0.025`) with dithering off by default.

You said:

Ah ok! That the quake palette consists of different ramps is known. Quake wiki: https://quakewiki.org/wiki/Quake_palette

here is your output
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga/tests$ sed -n '31,224p' palette_data.txt
30: RGB (131, 103, 35) H= 42.5 S=0.73 V=0.51
31: RGB (143, 111, 35) H= 42.2 S=0.76 V=0.56
32: RGB (11, 11, 15) H= 240.0 S=0.27 V=0.06
33: RGB (19, 19, 27) H= 240.0 S=0.30 V=0.11
34: RGB (27, 27, 39) H= 240.0 S=0.31 V=0.15
35: RGB (39, 39, 51) H= 240.0 S=0.24 V=0.20
36: RGB (47, 47, 63) H= 240.0 S=0.25 V=0.25
37: RGB (55, 55, 75) H= 240.0 S=0.27 V=0.29
38: RGB (63, 63, 87) H= 240.0 S=0.28 V=0.34
39: RGB (71, 71, 103) H= 240.0 S=0.31 V=0.40
40: RGB (79, 79, 115) H= 240.0 S=0.31 V=0.45
41: RGB (91, 91, 127) H= 240.0 S=0.28 V=0.50
42: RGB (99, 99, 139) H= 240.0 S=0.29 V=0.55
43: RGB (107, 107, 151) H= 240.0 S=0.29 V=0.59
44: RGB (115, 115, 163) H= 240.0 S=0.29 V=0.64
45: RGB (123, 123, 175) H= 240.0 S=0.30 V=0.69
46: RGB (131, 131, 187) H= 240.0 S=0.30 V=0.73
47: RGB (139, 139, 203) H= 240.0 S=0.32 V=0.80
48: RGB (0, 0, 0) H= 0.0 S=0.00 V=0.00
49: RGB (7, 7, 0) H= 60.0 S=1.00 V=0.03
50: RGB (11, 11, 0) H= 60.0 S=1.00 V=0.04
51: RGB (19, 19, 0) H= 60.0 S=1.00 V=0.07
52: RGB (27, 27, 0) H= 60.0 S=1.00 V=0.11
53: RGB (35, 35, 0) H= 60.0 S=1.00 V=0.14
54: RGB (43, 43, 7) H= 60.0 S=0.84 V=0.17
55: RGB (47, 47, 7) H= 60.0 S=0.85 V=0.18
56: RGB (55, 55, 7) H= 60.0 S=0.87 V=0.22
57: RGB (63, 63, 7) H= 60.0 S=0.89 V=0.25
58: RGB (71, 71, 7) H= 60.0 S=0.90 V=0.28
59: RGB (75, 75, 11) H= 60.0 S=0.85 V=0.29
60: RGB (83, 83, 11) H= 60.0 S=0.87 V=0.33
61: RGB (91, 91, 11) H= 60.0 S=0.88 V=0.36
62: RGB (99, 99, 11) H= 60.0 S=0.89 V=0.39
63: RGB (107, 107, 15) H= 60.0 S=0.86 V=0.42
64: RGB (7, 0, 0) H= 0.0 S=1.00 V=0.03
65: RGB (15, 0, 0) H= 0.0 S=1.00 V=0.06
66: RGB (23, 0, 0) H= 0.0 S=1.00 V=0.09
67: RGB (31, 0, 0) H= 0.0 S=1.00 V=0.12
68: RGB (39, 0, 0) H= 0.0 S=1.00 V=0.15
69: RGB (47, 0, 0) H= 0.0 S=1.00 V=0.18
70: RGB (55, 0, 0) H= 0.0 S=1.00 V=0.22

71: RGB (63, 0, 0) H= 0.0 S=1.00 V=0.25
72: RGB (71, 0, 0) H= 0.0 S=1.00 V=0.28
73: RGB (79, 0, 0) H= 0.0 S=1.00 V=0.31
74: RGB (87, 0, 0) H= 0.0 S=1.00 V=0.34
75: RGB (95, 0, 0) H= 0.0 S=1.00 V=0.37
76: RGB (103, 0, 0) H= 0.0 S=1.00 V=0.40
77: RGB (111, 0, 0) H= 0.0 S=1.00 V=0.44
78: RGB (119, 0, 0) H= 0.0 S=1.00 V=0.47
79: RGB (127, 0, 0) H= 0.0 S=1.00 V=0.50
80: RGB (19, 19, 0) H= 60.0 S=1.00 V=0.07
81: RGB (27, 27, 0) H= 60.0 S=1.00 V=0.11
82: RGB (35, 35, 0) H= 60.0 S=1.00 V=0.14
83: RGB (47, 43, 0) H= 54.9 S=1.00 V=0.18
84: RGB (55, 47, 0) H= 51.3 S=1.00 V=0.22
85: RGB (67, 55, 0) H= 49.3 S=1.00 V=0.26
86: RGB (75, 59, 7) H= 45.9 S=0.91 V=0.29
87: RGB (87, 67, 7) H= 45.0 S=0.92 V=0.34
88: RGB (95, 71, 7) H= 43.6 S=0.93 V=0.37
89: RGB (107, 75, 11) H= 40.0 S=0.90 V=0.42
90: RGB (119, 83, 15) H= 39.2 S=0.87 V=0.47
91: RGB (131, 87, 19) H= 36.4 S=0.85 V=0.51
92: RGB (139, 91, 19) H= 36.0 S=0.86 V=0.55
93: RGB (151, 95, 27) H= 32.9 S=0.82 V=0.59
94: RGB (163, 99, 31) H= 30.9 S=0.81 V=0.64
95: RGB (175, 103, 35) H= 29.1 S=0.80 V=0.69
96: RGB (35, 19, 7) H= 25.7 S=0.80 V=0.14
97: RGB (47, 23, 11) H= 20.0 S=0.77 V=0.18
98: RGB (59, 31, 15) H= 21.8 S=0.75 V=0.23
99: RGB (75, 35, 19) H= 17.1 S=0.75 V=0.29
100: RGB (87, 43, 23) H= 18.8 S=0.74 V=0.34
101: RGB (99, 47, 31) H= 14.1 S=0.69 V=0.39
102: RGB (115, 55, 35) H= 15.0 S=0.70 V=0.45
103: RGB (127, 59, 43) H= 11.4 S=0.66 V=0.50
104: RGB (143, 67, 51) H= 10.4 S=0.64 V=0.56
105: RGB (159, 79, 51) H= 15.6 S=0.68 V=0.62
106: RGB (175, 99, 47) H= 24.4 S=0.73 V=0.69
107: RGB (191, 119, 47) H= 30.0 S=0.75 V=0.75
108: RGB (207, 143, 43) H= 36.6 S=0.79 V=0.81
109: RGB (223, 171, 39) H= 43.0 S=0.83 V=0.87
110: RGB (239, 203, 31) H= 49.6 S=0.87 V=0.94
111: RGB (255, 243, 27) H= 56.8 S=0.89 V=1.00
112: RGB (11, 7, 0) H= 38.2 S=1.00 V=0.04
113: RGB (27, 19, 0) H= 42.2 S=1.00 V=0.11
114: RGB (43, 35, 15) H= 42.9 S=0.65 V=0.17
115: RGB (55, 43, 19) H= 40.0 S=0.65 V=0.22
116: RGB (71, 51, 27) H= 32.7 S=0.62 V=0.28
117: RGB (83, 55, 35) H= 25.0 S=0.58 V=0.33
118: RGB (99, 63, 43) H= 21.4 S=0.57 V=0.39
119: RGB (111, 71, 51) H= 20.0 S=0.54 V=0.44
120: RGB (127, 83, 63) H= 18.8 S=0.50 V=0.50
121: RGB (139, 95, 71) H= 21.2 S=0.49 V=0.55
122: RGB (155, 107, 83) H= 20.0 S=0.46 V=0.61
123: RGB (167, 123, 95) H= 23.3 S=0.43 V=0.65
124: RGB (183, 135, 107) H= 22.1 S=0.42 V=0.72
125: RGB (195, 147, 123) H= 20.0 S=0.37 V=0.76
126: RGB (211, 163, 139) H= 20.0 S=0.34 V=0.83
127: RGB (227, 179, 151) H= 22.1 S=0.33 V=0.89
128: RGB (171, 139, 163) H= 315.0 S=0.19 V=0.67

129: RGB (159, 127, 151) H= 315.0 S=0.20 V=0.62
130: RGB (147, 115, 135) H= 322.5 S=0.22 V=0.58
131: RGB (139, 103, 123) H= 326.7 S=0.26 V=0.55
132: RGB (127, 91, 111) H= 326.7 S=0.28 V=0.50
133: RGB (119, 83, 99) H= 333.3 S=0.30 V=0.47
134: RGB (107, 75, 87) H= 337.5 S=0.30 V=0.42
135: RGB (95, 63, 75) H= 337.5 S=0.34 V=0.37
136: RGB (87, 55, 67) H= 337.5 S=0.37 V=0.34
137: RGB (75, 47, 55) H= 342.9 S=0.37 V=0.29
138: RGB (67, 39, 47) H= 342.9 S=0.42 V=0.26
139: RGB (55, 31, 35) H= 350.0 S=0.44 V=0.22
140: RGB (43, 23, 27) H= 348.0 S=0.47 V=0.17
141: RGB (35, 19, 19) H= 0.0 S=0.46 V=0.14
142: RGB (23, 11, 11) H= 0.0 S=0.52 V=0.09
143: RGB (15, 7, 7) H= 0.0 S=0.53 V=0.06
144: RGB (187, 115, 159) H= 323.3 S=0.39 V=0.73
145: RGB (175, 107, 143) H= 328.2 S=0.39 V=0.69
146: RGB (163, 95, 131) H= 328.2 S=0.42 V=0.64
147: RGB (151, 87, 119) H= 330.0 S=0.42 V=0.59
148: RGB (139, 79, 107) H= 332.0 S=0.43 V=0.55
149: RGB (127, 75, 95) H= 336.9 S=0.41 V=0.50
150: RGB (115, 67, 83) H= 340.0 S=0.42 V=0.45
151: RGB (107, 59, 75) H= 340.0 S=0.45 V=0.42
152: RGB (95, 51, 63) H= 343.6 S=0.46 V=0.37
153: RGB (83, 43, 55) H= 342.0 S=0.48 V=0.33
154: RGB (71, 35, 43) H= 346.7 S=0.51 V=0.28
155: RGB (59, 31, 35) H= 351.4 S=0.47 V=0.23
156: RGB (47, 23, 27) H= 350.0 S=0.51 V=0.18
157: RGB (35, 19, 19) H= 0.0 S=0.46 V=0.14
158: RGB (23, 11, 11) H= 0.0 S=0.52 V=0.09
159: RGB (15, 7, 7) H= 0.0 S=0.53 V=0.06
160: RGB (219, 195, 187) H= 15.0 S=0.15 V=0.86
161: RGB (203, 179, 167) H= 20.0 S=0.18 V=0.80
162: RGB (191, 163, 155) H= 13.3 S=0.19 V=0.75
163: RGB (175, 151, 139) H= 20.0 S=0.21 V=0.69
164: RGB (163, 135, 123) H= 18.0 S=0.25 V=0.64
165: RGB (151, 123, 111) H= 18.0 S=0.26 V=0.59
166: RGB (135, 111, 95) H= 24.0 S=0.30 V=0.53
167: RGB (123, 99, 83) H= 24.0 S=0.33 V=0.48
168: RGB (107, 87, 71) H= 26.7 S=0.34 V=0.42
169: RGB (95, 75, 59) H= 26.7 S=0.38 V=0.37
170: RGB (83, 63, 51) H= 22.5 S=0.39 V=0.33
171: RGB (67, 51, 39) H= 25.7 S=0.42 V=0.26
172: RGB (55, 43, 31) H= 30.0 S=0.44 V=0.22
173: RGB (39, 31, 23) H= 30.0 S=0.41 V=0.15
174: RGB (27, 19, 15) H= 20.0 S=0.44 V=0.11
175: RGB (15, 11, 7) H= 30.0 S=0.53 V=0.06
176: RGB (111, 131, 123) H= 156.0 S=0.15 V=0.51
177: RGB (103, 123, 111) H= 144.0 S=0.16 V=0.48
178: RGB (95, 115, 103) H= 144.0 S=0.17 V=0.45
179: RGB (87, 107, 95) H= 144.0 S=0.19 V=0.42
180: RGB (79, 99, 87) H= 144.0 S=0.20 V=0.39
181: RGB (71, 91, 79) H= 144.0 S=0.22 V=0.36
182: RGB (63, 83, 71) H= 144.0 S=0.24 V=0.33
183: RGB (55, 75, 63) H= 144.0 S=0.27 V=0.29
184: RGB (47, 67, 55) H= 144.0 S=0.30 V=0.26
185: RGB (43, 59, 47) H= 135.0 S=0.27 V=0.23
186: RGB (35, 51, 39) H= 135.0 S=0.31 V=0.20

187: RGB (31, 43, 31) H= 120.0 S=0.28 V=0.17
188: RGB (23, 35, 23) H= 120.0 S=0.34 V=0.14
189: RGB (15, 27, 19) H= 140.0 S=0.44 V=0.11
190: RGB (11, 19, 11) H= 120.0 S=0.42 V=0.07
191: RGB (7, 11, 7) H= 120.0 S=0.36 V=0.04
192: RGB (255, 243, 27) H= 56.8 S=0.89 V=1.00
193: RGB (239, 223, 23) H= 55.6 S=0.90 V=0.94
194: RGB (219, 203, 19) H= 55.2 S=0.91 V=0.86
195: RGB (203, 183, 15) H= 53.6 S=0.93 V=0.80
196: RGB (187, 167, 15) H= 53.0 S=0.92 V=0.73
197: RGB (171, 151, 11) H= 52.5 S=0.94 V=0.67
198: RGB (155, 131, 7) H= 50.3 S=0.95 V=0.61
199: RGB (139, 115, 7) H= 49.1 S=0.95 V=0.55
200: RGB (123, 99, 7) H= 47.6 S=0.94 V=0.48
201: RGB (107, 83, 0) H= 46.5 S=1.00 V=0.42
202: RGB (91, 71, 0) H= 46.8 S=1.00 V=0.36
203: RGB (75, 55, 0) H= 44.0 S=1.00 V=0.29
204: RGB (59, 43, 0) H= 43.7 S=1.00 V=0.23
205: RGB (43, 31, 0) H= 43.3 S=1.00 V=0.17
206: RGB (27, 15, 0) H= 33.3 S=1.00 V=0.11
207: RGB (11, 7, 0) H= 38.2 S=1.00 V=0.04
208: RGB (0, 0, 255) H= 240.0 S=1.00 V=1.00
209: RGB (11, 11, 239) H= 240.0 S=0.95 V=0.94
210: RGB (19, 19, 223) H= 240.0 S=0.91 V=0.87
211: RGB (27, 27, 207) H= 240.0 S=0.87 V=0.81
212: RGB (35, 35, 191) H= 240.0 S=0.82 V=0.75
213: RGB (43, 43, 175) H= 240.0 S=0.75 V=0.69
214: RGB (47, 47, 159) H= 240.0 S=0.70 V=0.62
215: RGB (47, 47, 143) H= 240.0 S=0.67 V=0.56
216: RGB (47, 47, 127) H= 240.0 S=0.63 V=0.50
217: RGB (47, 47, 111) H= 240.0 S=0.58 V=0.44
218: RGB (47, 47, 95) H= 240.0 S=0.51 V=0.37
219: RGB (43, 43, 79) H= 240.0 S=0.46 V=0.31
220: RGB (35, 35, 63) H= 240.0 S=0.44 V=0.25
221: RGB (27, 27, 47) H= 240.0 S=0.43 V=0.18
222: RGB (19, 19, 31) H= 240.0 S=0.39 V=0.12
223: RGB (11, 11, 15) H= 240.0 S=0.27 V=0.06\

Worked for 10s

ChatGPT said:

You're right—the ramp structure is already documented, and your data confirms that `quake.pal` is the standard Quake palette layout. We don't need to rediscover the ramps from scratch. The useful work is deciding **how to map Soldier of Fortune texture colors onto those ramps** without introducing speckling or flattening detail.

A few important observations from your listing:

- **0–15:** grayscale ramp.
- **16–31:** dark brown/ochre ramp.
- **32–47:** blue-grey ramp.
- **48–63:** dark yellow-green ramp.
- **64–79:** red ramp.
- **80–95:** yellow-to-orange/brown ramp.
- **96–111:** reddish brown through bright yellow.
- **112–127:** muted brown/tan ramp.
- **128–159:** two muted purple/red ramps.
- **160–175:** light-to-dark desaturated brown.
- **176–191:** green-grey ramp.

- **192–207:** yellow/ochre ramp.
- **208–223:** saturated blue ramp.

The important distinction is that these ramps aren't interchangeable just because some colors are close in RGB or hue. For example, the blue-grey ramp (32–47) and saturated blue ramp (208–223) have very different character and tonal ranges.

For the military/industrial textures you're converting, I'd prioritize these families:

Purpose	Palette indices
Neutral shadows and highlights	0–15
Dark earth / ochre	16–31
Blue-grey shadows	32–47
Olive / dark yellow-green	48–63
Warm earth / orange-brown	80–95
Reddish earth	96–111
Muted earth / tan	112–127
Desaturated stone / brown	160–175
Green-grey	176–191
Yellow-ochre	192–207

I'd leave the red and purple ramps out of the normal candidate set initially, and retain the blue ramps only where the texture genuinely benefits from blue-grey shading.

The conversion strategy I'd use:

1. Lightly smooth the source (`0.025` , as before).
2. Analyze the texture's overall color distribution to choose suitable palette families.
3. Map pixels to those families, rather than allowing every pixel to jump independently between unrelated ramps.
4. Within each selected ramp, preserve brightness ordering so surface details and shading survive.
5. Allow controlled hue shifts—especially blue-grey toward green-grey and ambiguous brown toward olive—when that produces a more convincing Quake-era texture.
6. Keep dithering off and exclude indices 224–255 from all normal color selection.

One thing I need before writing the replacement: **please paste your current** `conv.py` (or attach it). I want to preserve the working TGA/PNG handling and command-line options, while replacing the problematic ramp-selection logic rather than giving you another script that may not match your setup.

You said:

the conv.py is exactly what you gave me last time.

ChatGPT said:

Understood. No need to paste it again. I'll work from the `conv.py` version we established earlier.

The next version will replace the automatic 12-sector hue grouping with a **palette-ramp-aware quantizer**. I'll retain:

- Your existing TGA input and PNG output handling.
- The current command-line workflow.
- Light smoothing (`0.025`) as the default.
- No dithering by default.
- Fullbright exclusion (indices 224–255).
- The original 256-entry palette in the output PNG's palette table.

I'll also make the palette-family selection adjustable, so we can compare olive, green-grey, blue-grey and earth-tone mappings without repeatedly editing the code.

One important correction to my previous description: indices 192–207 are a yellow/ochre ramp, not a green ramp. We'll use 48–63 and 176–191 for the greener mappings.

You said:

give it to me

ChatGPT said:

Replace your existing `conv.py` with the version below. This uses the actual Quake palette ramps, excludes fullbrights from quantization, and avoids per-pixel switching between unrelated color families.

The defaults are deliberately conservative: light smoothing, no dithering, and a broad selection of earth, olive, green-grey and blue-grey ramps.

```
#!/usr/bin/env python3

import argparse
import colorsys
import sys
from pathlib import Path

import numpy as np
from PIL import Image
from scipy.ndimage import gaussian_filter, median_filter


# ------------------------------------------------------------
# Quake palette ramp definitions
# ------------------------------------------------------------

RAMPS = {
    "neutral": list(range(0, 16)),
    "dark_earth": list(range(16, 32)),
    "blue_grey": list(range(32, 48)),
    "olive_dark": list(range(48, 64)),
    "red": list(range(64, 80)),
    "ochre": list(range(80, 96)),
    "warm_earth": list(range(96, 112)),
    "tan": list(range(112, 128)),
    "purple_dark": list(range(128, 144)),
    "purple_warm": list(range(144, 160)),
    "stone": list(range(160, 176)),
    "green_grey": list(range(176, 192)),
    "yellow_ochre": list(range(192, 208)),
    "blue": list(range(208, 224)),
}

DEFAULT_RAMPS = [
    "neutral",
    "dark_earth",
    "blue_grey",
    "olive_dark",
    "ochre",
    "warm_earth",
    "tan",
    "stone",
    "green_grey",
```

```
]

# ------------------------------------------------------------
# Palette loading
# ------------------------------------------------------------

def load_palette(filename):
    """Load a JASC-PAL palette."""
    path = Path(filename)

    with path.open("r", encoding="ascii") as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 3 or lines[0] != "JASC-PAL":
        raise ValueError(f"{filename} is not a valid JASC-PAL file")

    count = int(lines[2])
    colors = []

    for line in lines[3:3 + count]:
        values = [int(x) for x in line.split()]
        if len(values) != 3:
            raise ValueError(f"Invalid palette entry: {line}")
        colors.append(values)

    if len(colors) != count:
        raise ValueError("Palette entry count does not match header")

    return np.asarray(colors, dtype=np.uint8)


# ------------------------------------------------------------
# Color conversion
# ------------------------------------------------------------

def srgb_to_linear(rgb):
    rgb = np.asarray(rgb, dtype=np.float32) / 255.0
    return np.where(
        rgb <= 0.04045,
        rgb / 12.92,
        ((rgb + 0.055) / 1.055) ** 2.4
    )


def linear_to_oklab(rgb):
    rgb = srgb_to_linear(rgb)

    l = (
        0.4122214708 * rgb[..., 0]
        + 0.5363325363 * rgb[..., 1]
        + 0.0514459929 * rgb[..., 2]
    )
    m = (
        0.2119034982 * rgb[..., 0]
        + 0.6806995451 * rgb[..., 1]
        + 0.1073969566 * rgb[..., 2]
    )
    s = (
        0.0883024619 * rgb[..., 0]
        + 0.2817188376 * rgb[..., 1]
        + 0.6299787005 * rgb[..., 2]
    )
```

```
        l_ = np.cbrt(l)
        m_ = np.cbrt(m)
        s_ = np.cbrt(s)

        return np.stack([
            0.2104542553 * l_ + 0.7936177850 * m_ - 0.0040720468 * s_,
            1.9779984951 * l_ - 2.4285922050 * m_ + 0.4505937099 * s_,
            0.0259040371 * l_ + 0.7827717662 * m_ - 0.8086757660 * s_,
        ], axis=-1)


    # ------------------------------------------------------------
    # Image preprocessing
    # ------------------------------------------------------------

    def smooth_image(image, amount=0.025):
        """
        Very light edge-preserving smoothing.

        amount 0 disables smoothing.
        The default is intentionally subtle.
        """
        if amount <= 0:
            return image.astype(np.float32)

        image = image.astype(np.float32)

        # Small bilateral-like approximation:
        # blend a lightly blurred image with the original.
        blurred = gaussian_filter(
            image,
            sigma=(0.65, 0.65, 0),
            mode="nearest"
        )

        # Preserve edges by limiting the blend according to local difference.
        difference = np.mean(np.abs(image - blurred), axis=2)
        edge_weight = np.exp(-difference / 24.0)

        blend = amount * 4.0
        blend = min(blend, 0.25)

        weight = blend * edge_weight[..., None]

        return image * (1.0 - weight) + blurred * weight


    # ------------------------------------------------------------
    # Image-level palette family selection
    # ------------------------------------------------------------

    def analyze_image(image):
        """
        Estimate broad source color characteristics.

        Returns average hue, saturation and brightness.
        """
        pixels = image.reshape(-1, 3).astype(np.float32)

        # Sample large images for speed.
        if len(pixels) > 50000:
            indices = np.linspace(
                0, len(pixels) - 1, 50000
```

```
            ).astype(int)
            pixels = pixels[indices]

        hsv = np.array([
            colorsys.rgb_to_hsv(*(p / 255.0))
            for p in pixels
        ])

        # Circular mean for hue.
        angles = hsv[:, 0] * 2.0 * np.pi
        hue = np.arctan2(
            np.mean(np.sin(angles)),
            np.mean(np.cos(angles))
        ) / (2.0 * np.pi)

        hue %= 1.0

        return (
            float(hue),
            float(np.mean(hsv[:, 1])),
            float(np.mean(hsv[:, 2]))
        )


    def choose_ramps(image, requested=None):
        """
        Choose palette families based on the image as a whole.

        The default is intentionally broad enough for military,
        industrial, stone and terrain textures.
        """
        if requested:
            names = requested.split(",")
            for name in names:
                if name not in RAMPS:
                    raise ValueError(f"Unknown ramp: {name}")
            return names

        hue, saturation, brightness = analyze_image(image)

        ramps = list(DEFAULT_RAMPS)

        # Blue-heavy source textures benefit from blue-grey entries.
        if 0.48 <= hue <= 0.75:
            if "blue_grey" not in ramps:
                ramps.append("blue_grey")

        # Green-heavy images should retain the green-grey ramp.
        if 0.20 <= hue <= 0.48:
            if "green_grey" not in ramps:
                ramps.append("green_grey")

        return ramps


    # ------------------------------------------------------------
    # Quantization
    # ------------------------------------------------------------

    def build_candidates(palette, ramp_names, fullbright_start=224):
        indices = []

        for name in ramp_names:
```

```
            indices.extend(RAMPS[name])

        indices = sorted(set(
            i for i in indices
            if 0 <= i < min(fullbright_start, len(palette))
        ))

        if not indices:
            raise ValueError("No valid palette colors selected")

        return np.asarray(indices, dtype=np.int32), palette[indices]


    def quantize(image, palette, ramp_names, strength=1.0,
                 fullbright_start=224):
        """
        Quantize using selected Quake ramps.

        OKLab distance is used for perceptual matching.
        A small brightness penalty discourages noisy jumps
        between very different tonal levels.
        """
        indices, candidate_rgb = build_candidates(
            palette, ramp_names, fullbright_start
        )

        source = image.reshape(-1, 3).astype(np.float32)

        source_lab = linear_to_oklab(source)
        palette_lab = linear_to_oklab(candidate_rgb)

        output = np.empty(len(source), dtype=np.uint8)

        # Process in chunks to limit memory usage.
        chunk_size = 8192

        for start in range(0, len(source), chunk_size):
            end = min(start + chunk_size, len(source))

            src = source_lab[start:end]

            delta = src[:, None, :] - palette_lab[None, :, :]

            # Standard perceptual distance.
            distance = np.sum(delta * delta, axis=2)

            # Discourage large brightness changes.
            brightness_delta = (
                src[:, None, 0] - palette_lab[None, :, 0]
            )

            distance += 0.08 * brightness_delta * brightness_delta

            # Slightly reduce chroma pressure to avoid noisy colors.
            distance[:, :] *= strength

            best = np.argmin(distance, axis=1)

            output[start:end] = indices[best]

        return output.reshape(image.shape[:2])
```

```
# ------------------------------------------------------------
# PNG palette output
# ------------------------------------------------------------

def save_indexed_png(indices, palette, output_path):
    """
    Save indexed PNG with the full 256-entry palette table.

    Only indices below fullbright_start are selected during
    quantization. The full palette is retained in the PNG.
    """
    image = Image.fromarray(indices.astype(np.uint8), mode="P")

    flat_palette = palette.astype(np.uint8).flatten().tolist()

    # PNG palette tables contain 256 RGB entries.
    flat_palette += [0] * (768 - len(flat_palette))
    image.putpalette(flat_palette[:768])

    image.save(output_path, optimize=False)


# ------------------------------------------------------------
# Main
# ------------------------------------------------------------

def main():
    parser = argparse.ArgumentParser(
        description="Convert textures to the Quake 256-color palette"
    )

    parser.add_argument("input", help="Input TGA or image")
    parser.add_argument("output", help="Output indexed PNG")

    parser.add_argument(
        "--palette",
        default="quake.pal",
        help="JASC-PAL palette file (default: quake.pal)"
    )

    parser.add_argument(
        "--smooth",
        type=float,
        default=0.025,
        help="Smoothing amount (default: 0.025, 0 disables)"
    )

    parser.add_argument(
        "--ramps",
        default=None,
        help="Comma-separated palette ramps to use"
    )

    parser.add_argument(
        "--fullbright-start",
        type=int,
        default=224,
        help="First fullbright palette index (default: 224)"
    )

    parser.add_argument(
        "--strength",
        type=float,
```

```
            default=1.0,
            help="Color matching multiplier (default: 1.0)"
        )

        args = parser.parse_args()

        try:
            palette = load_palette(args.palette)

            if len(palette) != 256:
                raise ValueError(
                    f"Expected 256 palette entries, got {len(palette)}"
                )

            if not 1 <= args.fullbright_start <= 256:
                raise ValueError("--fullbright-start must be between 1 and 256")

            source = Image.open(args.input).convert("RGB")
            image = np.asarray(source, dtype=np.float32)

            image = smooth_image(image, args.smooth)

            ramp_names = choose_ramps(image, args.ramps)

            print("Using ramps:", ", ".join(ramp_names))
            print(
                f"Normal palette indices: 0-{args.fullbright_start - 1}"
            )

            result = quantize(
                image,
                palette,
                ramp_names,
                args.strength,
                args.fullbright_start
            )

            save_indexed_png(result, palette, args.output)

            used = np.unique(result)
            print(f"Saved: {args.output}")
            print(f"Colors used: {len(used)}")
            print(f"Palette indices used: {used.min()}-{used.max()}")

        except Exception as exc:
            print(f"Error: {exc}", file=sys.stderr)
            sys.exit(1)


    if __name__ == "__main__":
        main()
```

First test

Run the same texture as before:

```
.venv/bin/python conv.py f_floortile2.tga output.png
```

You can also test a more olive-focused mapping:

```
.venv/bin/python conv.py f_floortile2.tga output_olive.png --ramps
neutral,dark_earth,olive_dark,ochre,warm_earth,tan,stone,green_grey
```

And a mapping with more blue-grey:

```
.venv/bin/python conv.py f_floortile2.tga output_bluegrey.png --ramps
neutral,dark_earth,blue_grey,olive_dark,ochre,warm_earth,tan,stone,green_grey
```

One technical note: this is a first ramp-aware baseline, not yet a spatially coherent ramp mapper. It restricts the candidate palette to selected ramps, but individual pixels can still choose different ramps. The next improvement should address that if speckling remains. Also, the `--strength` option currently scales all distances equally, so changing it won't materially change the selected colors; it is retained for compatibility and should not be treated as a hue or saturation control.

You said:

the three versions look identical. A bit too smooth but ok compared to most what came before.

ChatGPT said:

That result makes sense—and it exposes a flaw in the current implementation.

The three versions look identical because the default ramp set already includes `blue_grey`, `olive_dark`, `ochre`, `warm_earth`, `tan`, `stone` and `green_grey`. Your two test commands therefore selected almost exactly the same colors.

There are two things I'd change next:

1. **Reduce smoothing:** change the default from `0.025` to `0.01`. The current implementation is a blur blend, not a true edge-preserving bilateral filter, so even small amounts can soften fine texture.
2. **Make ramp selection meaningful:** instead of giving every pixel access to all selected ramps, analyze the image and assign dominant color regions to specific ramps. That should make olive versus blue-grey variants visibly different and reduce speckling caused by unrelated ramps competing locally.

I would not change the color-distance metric yet. The current output is a useful baseline, and we should isolate the effect of ramp selection before tuning color matching further.

One question: Is the remaining smoothness mainly losing fine surface detail (scratches, cracks, grain), or does the whole texture look slightly blurred?

You said:

is loses low contrast surface detail resulting in a bit of spotty look.

ChatGPT said:

That's an important distinction. The problem isn't necessarily excessive blur; it's that **low-contrast texture variations are being quantized away**, leaving isolated patches of similar colors. That creates the spotty appearance.

I'd change the algorithm in three ways:

1. **Preserve local contrast:** avoid smoothing the source image. Set smoothing to `0` by default.
2. **Preserve tonal transitions:** add a local contrast term to the color-matching process, so subtle changes in surface brightness are less likely to collapse into the same palette entry.
3. **Use coherent ramps:** select a dominant palette family for a region, then use neighboring shades within that ramp. This should reduce isolated color patches.

I would not add dithering yet. It might disguise the spots, but it can also introduce the noise we have been trying to eliminate.

For the next iteration, I'd start with `--smooth 0` and preserve more of the source's local luminance variation. The goal is a slightly grainier but more detailed Quake-style texture, rather than a smooth image with scattered color patches.

You said:

just a concept question: What about shape detection stuff?

ChatGPT said:

Shape detection could actually be a very promising direction. In fact, it addresses a weakness of ordinary color quantization: **a texture isn't just a collection of colors; it contains surfaces, edges, material regions and patterns.**

Instead of treating every pixel independently, we could make the converter somewhat aware of the structures in the texture.

What kind of shape detection?

I'd consider four techniques, each with a different purpose.

1. Edge detection

Detect boundaries between structural features:

- Tile edges
- Cracks
- Scratches
- Panel boundaries
- Surface damage

The quantizer could protect these edges from smoothing and preserve their contrast.

My assessment: Very useful. Probably the first technique I'd implement.

2. Region segmentation

Divide the texture into coherent regions, or superpixels.

For example, a floor tile might contain:

- Main stone surface
- Dark grout
- Scratches
- Dirt patches
- Highlighted edges

Each region could receive a consistent palette-family assignment.

This would prevent neighboring pixels from randomly switching between brown, olive and grey ramps.

My assessment: Probably the most important technique for reducing the spotty appearance.

3. Texture feature detection

Analyze local texture characteristics:

- Fine grain
- Roughness
- Directional scratches
- Repeating patterns
- Smooth surfaces

This would let the converter treat a smooth concrete surface differently from a noisy, weathered metal surface.

My assessment: Interesting for a later version, especially if you want automatic material-aware conversion.

4. Shape-aware palette mapping

This is where the techniques come together.

Imagine a texture containing a large concrete slab:

- The main surface is mapped to the green-grey or stone ramp.
- Dark cracks remain dark and continuous.
- Small highlights remain highlights.
- Dirt patches use earth tones.
- Edges retain their contrast.

The important thing is that color decisions become aware of the underlying structure.

My proposed architecture

I'd build the converter as a pipeline:

Original TGA texture

↓

Structure analysis
Edges · regions · local contrast

↓

Material and palette-family assignment
Earth · olive · stone · green-grey · blue-grey

↓

Shape-aware color quantization
Preserve edges, tonal detail and coherent shading

↓

Quake indexed PNG
224 normal colors · fullbright entries untouched

What I'd actually implement first

I'd start with **SLIC superpixels + edge-aware quantization**.

SLIC (Simple Linear Iterative Clustering) groups pixels into compact, visually coherent regions. It is relatively lightweight and available through `scikit-image`.

The advantage is that we don't need sophisticated AI or neural networks. Classical image processing should be sufficient for this task.

One important caveat: superpixels must not become visible blocks. They should guide palette selection, not force every pixel in a region to the same color.

My expectation is that this approach could preserve much more of the original surface character while producing the restrained, coherent color distribution associated with Quake textures.

You said:

ok1 Put it all in!

Worked for 15s

ChatGPT said:

Let's build the more ambitious version. This replaces `conv.py` with a structure-aware converter using **SLIC superpixels, edge detection, local contrast preservation, and coherent palette-ramp assignment**.

The important design decision: segmentation guides the color mapping, but it does not flatten each region into one color. Individual pixels still receive their own palette entries, preserving scratches, grain, cracks and shading.

1. Install the additional dependency

From your `tests` directory:

```
.venv/bin/pip install scikit-image
```

2. Replace `conv.py`

```
#!/usr/bin/env python3

import argparse
import sys
from pathlib import Path

import numpy as np
from PIL import Image

from scipy.ndimage import gaussian_filter
from skimage.segmentation import slic
from skimage.filters import sobel


# ============================================================
# QUAKE PALETTE RAMPS
# ============================================================
```

```
RAMPS = {
    "neutral":       list(range(0, 16)),
    "dark_earth":    list(range(16, 32)),
    "blue_grey":     list(range(32, 48)),
    "olive_dark":    list(range(48, 64)),
    "red":           list(range(64, 80)),
    "ochre":         list(range(80, 96)),
    "warm_earth":    list(range(96, 112)),
    "tan":           list(range(112, 128)),
    "purple_dark":   list(range(128, 144)),
    "purple_warm":   list(range(144, 160)),
    "stone":         list(range(160, 176)),
    "green_grey":    list(range(176, 192)),
    "yellow_ochre":  list(range(192, 208)),
    "blue":          list(range(208, 224)),
}

DEFAULT_RAMPS = [
    "neutral",
    "dark_earth",
    "blue_grey",
    "olive_dark",
    "ochre",
    "warm_earth",
    "tan",
    "stone",
    "green_grey",
]


# ============================================================
# PALETTE LOADING
# ============================================================

def load_palette(filename):
    path = Path(filename)

    with path.open("r", encoding="ascii") as f:
        lines = [line.strip() for line in f if line.strip()]

    if len(lines) < 3 or lines[0] != "JASC-PAL":
        raise ValueError("Not a valid JASC-PAL palette")

    count = int(lines[2])
    colors = []

    for line in lines[3:3 + count]:
        rgb = [int(x) for x in line.split()]
        if len(rgb) != 3:
            raise ValueError("Invalid palette entry")
        colors.append(rgb)

    if len(colors) != count:
        raise ValueError("Palette entry count mismatch")

    return np.asarray(colors, dtype=np.uint8)


# ============================================================
# COLOR SPACE: sRGB -> OKLab
# ============================================================
```

```
    def srgb_to_linear(rgb):
        rgb = np.asarray(rgb, dtype=np.float32) / 255.0

        return np.where(
            rgb <= 0.04045,
            rgb / 12.92,
            ((rgb + 0.055) / 1.055) ** 2.4
        )


    def to_oklab(rgb):
        rgb = srgb_to_linear(rgb)

        l = (
            0.4122214708 * rgb[..., 0]
            + 0.5363325363 * rgb[..., 1]
            + 0.0514459929 * rgb[..., 2]
        )

        m = (
            0.2119034982 * rgb[..., 0]
            + 0.6806995451 * rgb[..., 1]
            + 0.1073969566 * rgb[..., 2]
        )

        s = (
            0.0883024619 * rgb[..., 0]
            + 0.2817188376 * rgb[..., 1]
            + 0.6299787005 * rgb[..., 2]
        )

        l = np.cbrt(l)
        m = np.cbrt(m)
        s = np.cbrt(s)

        return np.stack([
            0.2104542553 * l + 0.7936177850 * m - 0.0040720468 * s,
            1.9779984951 * l - 2.4285922050 * m + 0.4505937099 * s,
            0.0259040371 * l + 0.7827717662 * m - 0.8086757660 * s,
        ], axis=-1)


    # ============================================================
    # LOCAL CONTRAST AND DETAIL PRESERVATION
    # ============================================================

    def enhance_detail(image, amount=0.35, radius=1.2):
        """
        Enhance low-contrast luminance detail.

        No denoising or general-purpose blur is applied.
        The high-frequency luminance component is restored
        to the source before palette quantization.
        """

        if amount <= 0:
            return image.astype(np.float32)

        lab = to_oklab(image)

        luminance = lab[..., 0]

        base = gaussian_filter(
```

```
            luminance,
            sigma=radius,
            mode="nearest"
        )

        detail = luminance - base

        # Restrict enhancement to subtle variations.
        # This avoids excessively amplifying strong edges.
        detail = np.clip(detail, -0.12, 0.12)

        lab[..., 0] += amount * detail

        # Convert adjusted OKLab back to approximate RGB.
        L = lab[..., 0]
        a = lab[..., 1]
        b = lab[..., 2]

        l_ = L + 0.3963377774 * a + 0.2158037573 * b
        m_ = L - 0.1055613458 * a - 0.0638541728 * b
        s_ = L - 0.0894841775 * a - 1.2914855480 * b

        l = l_ ** 3
        m = m_ ** 3
        s = s_ ** 3

        rgb_linear = np.stack([
            4.0767416621 * l - 3.3077115913 * m + 0.2309699292 * s,
            -1.2684380046 * l + 2.6097574011 * m - 0.3413193965 * s,
            -0.0041960863 * l - 0.7034186147 * m + 1.7076147010 * s,
        ], axis=-1)

        rgb = np.where(
            rgb_linear <= 0.0031308,
            rgb_linear * 12.92,
            1.055 * np.maximum(rgb_linear, 0) ** (1 / 2.4) - 0.055
        )

        return np.clip(rgb * 255.0, 0, 255).astype(np.float32)


    # ============================================================
    # EDGE DETECTION
    # ============================================================

    def detect_edges(image):
        """
        Calculate an edge-strength map.

        Used to protect structural detail during region
        analysis and ramp assignment.
        """

        gray = (
            0.299 * image[..., 0]
            + 0.587 * image[..., 1]
            + 0.114 * image[..., 2]
        ) / 255.0

        edges = sobel(gray)

        return edges.astype(np.float32)
```

```
# ============================================================
# SLIC SUPERPIXEL SEGMENTATION
# ============================================================

def segment_image(image, segments=180, compactness=10):
    """
    Divide the texture into coherent visual regions.

    Segmentation guides ramp selection but does not
    force pixels within a region to share one color.
    """

    labels = slic(
        image.astype(np.uint8),
        n_segments=segments,
        compactness=compactness,
        sigma=0,
        convert2lab=True,
        enforce_connectivity=True,
        start_label=0,
        channel_axis=-1
    )

    return labels.astype(np.int32)


# ============================================================
# SUPERPIXEL NEIGHBORHOOD GRAPH
# ============================================================

def build_adjacency(labels):
    adjacency = set()

    for a, b in [
        (labels[:, :-1], labels[:, 1:]),
        (labels[:-1, :], labels[1:, :])
    ]:
        mask = a != b

        pairs = np.stack([a[mask], b[mask]], axis=1)

        for x, y in pairs:
            adjacency.add((min(int(x), int(y)),
                           max(int(x), int(y))))

    return list(adjacency)


# ============================================================
# RAMP PREPARATION
# ============================================================

def get_ramps(palette, names, fullbright_start):
    result = {}

    for name in names:
        if name not in RAMPS:
            raise ValueError(f"Unknown ramp: {name}")

        indices = [
            i for i in RAMPS[name]
            if i < fullbright_start and i < len(palette)
```

```
            ]

            if indices:
                result[name] = np.asarray(indices, dtype=np.int32)

        if not result:
            raise ValueError("No usable palette ramps")

        return result


    # ============================================================
    # REGION COLOR ANALYSIS
    # ============================================================

    def region_statistics(image, labels, edges):
        count = int(labels.max()) + 1

        means = np.zeros((count, 3), dtype=np.float32)
        edge_means = np.zeros(count, dtype=np.float32)
        sizes = np.zeros(count, dtype=np.float32)

        for region in range(count):
            mask = labels == region

            pixels = image[mask]

            if len(pixels) == 0:
                continue

            means[region] = np.mean(pixels, axis=0)
            edge_means[region] = np.mean(edges[mask])
            sizes[region] = len(pixels)

        return means, edge_means, sizes


    # ============================================================
    # CHOOSE A RAMP FOR EACH REGION
    # ============================================================

    def assign_region_ramps(
        image,
        labels,
        edges,
        palette,
        ramps,
        coherence=0.025
    ):
        """
        Choose a suitable palette ramp for each superpixel.

        Uses:
          - Region-average color
          - Perceptual OKLab distance
          - Neighboring-region coherence

        Neighboring regions are encouraged to use related
        ramps, but strong color differences can override this.
        """

        names = list(ramps.keys())
```

```
means, edge_means, sizes = region_statistics(
    image, labels, edges
)

region_lab = to_oklab(means)

ramp_labs = {}

for name, indices in ramps.items():
    ramp_labs[name] = to_oklab(palette[indices])

region_count = len(means)

costs = np.zeros(
    (region_count, len(names)),
    dtype=np.float32
)

for region in range(region_count):
    for j, name in enumerate(names):
        candidates = ramp_labs[name]

        delta = candidates - region_lab[region]

        distance = np.sum(delta * delta, axis=1)

        # Find the best color within this ramp.
        costs[region, j] = np.min(distance)

adjacency = build_adjacency(labels)

# Initial independent assignments.
assignments = np.argmin(costs, axis=1)

# Iterative neighborhood optimization.
# Strong image boundaries reduce the coherence penalty.
for _ in range(5):
    changed = False

    for region in range(region_count):
        scores = costs[region].copy()

        for a, b in adjacency:
            if a == region:
                neighbor = b
            elif b == region:
                neighbor = a
            else:
                continue

            # Similar regions should tend to use
            # the same ramp family.
            difference = np.linalg.norm(
                region_lab[region] - region_lab[neighbor]
            )

            edge_strength = max(
                edge_means[region],
                edge_means[neighbor]
            )

            penalty = coherence * np.exp(
                -difference * 5.0
```

```
                )

                penalty *= max(0.0, 1.0 - edge_strength * 3.0)

                for j in range(len(names)):
                    if j != assignments[neighbor]:
                        scores[j] += penalty

            best = int(np.argmin(scores))

            if best != assignments[region]:
                assignments[region] = best
                changed = True

        if not changed:
            break

    return assignments, names


# ============================================================
# PIXEL-LEVEL QUANTIZATION
# ============================================================

def quantize_pixels(
    image,
    labels,
    assignments,
    names,
    ramps,
    palette,
    fullbright_start,
    edge_map
):
    """
    Quantize each pixel using the ramp assigned to
    its superpixel.

    Pixel-level matching retains the original texture
    detail and tonal variations.
    """

    height, width = image.shape[:2]

    output = np.zeros((height, width), dtype=np.uint8)

    # Cache palette colors in OKLab.
    ramp_data = {}

    for name, indices in ramps.items():
        colors = palette[indices]
        ramp_data[name] = (
            indices,
            colors,
            to_oklab(colors)
        )

    for region in range(int(labels.max()) + 1):
        mask = labels == region

        if not np.any(mask):
            continue
```

```
            name = names[assignments[region]]

            indices, colors, colors_lab = ramp_data[name]

            pixels = image[mask].astype(np.float32)
            pixels_lab = to_oklab(pixels)

            delta = (
                pixels_lab[:, None, :]
                - colors_lab[None, :, :]
            )

            distance = np.sum(delta * delta, axis=2)

            # Slight luminance emphasis preserves tonal structure.
            light_delta = (
                pixels_lab[:, None, 0]
                - colors_lab[None, :, 0]
            )

            distance += 0.08 * light_delta * light_delta

            best = np.argmin(distance, axis=1)

            output[mask] = indices[best]

        return output


    # ============================================================
    # SAVE INDEXED PNG
    # ============================================================

    def save_png(indices, palette, filename):
        image = Image.fromarray(indices.astype(np.uint8), mode="P")

        table = palette.flatten().tolist()
        table += [0] * (768 - len(table))

        image.putpalette(table[:768])
        image.save(filename, optimize=False)


    # ============================================================
    # MAIN
    # ============================================================

    def main():

        parser = argparse.ArgumentParser(
            description="Structure-aware Quake palette texture converter"
        )

        parser.add_argument("input")
        parser.add_argument("output")

        parser.add_argument(
            "--palette",
            default="quake.pal"
        )

        parser.add_argument(
            "--ramps",
```

```
        default=",".join(DEFAULT_RAMPS)
    )

    parser.add_argument(
        "--fullbright-start",
        type=int,
        default=224
    )

    parser.add_argument(
        "--segments",
        type=int,
        default=180,
        help="Number of SLIC regions (default: 180)"
    )

    parser.add_argument(
        "--compactness",
        type=float,
        default=10.0
    )

    parser.add_argument(
        "--detail",
        type=float,
        default=0.35,
        help="Local contrast enhancement (default: 0.35)"
    )

    parser.add_argument(
        "--detail-radius",
        type=float,
        default=1.2
    )

    parser.add_argument(
        "--coherence",
        type=float,
        default=0.025,
        help="Ramp-family neighborhood coherence"
    )

    args = parser.parse_args()

    try:
        palette = load_palette(args.palette)

        if len(palette) != 256:
            raise ValueError("Expected a 256-entry palette")

        if not 1 <= args.fullbright_start <= 256:
            raise ValueError("--fullbright-start must be 1..256")

        if args.segments < 1:
            raise ValueError("--segments must be positive")

        source = Image.open(args.input).convert("RGB")
        original = np.asarray(source, dtype=np.float32)

        print("Analyzing texture...")

        # No general-purpose smoothing.
        # Preserve original fine surface information.
```

```
    image = enhance_detail(
        original,
        amount=args.detail,
        radius=args.detail_radius
    )

    print("Detecting edges...")
    edges = detect_edges(image)

    print("Segmenting image...")
    labels = segment_image(
        image,
        segments=args.segments,
        compactness=args.compactness
    )

    ramp_names = [
        x.strip()
        for x in args.ramps.split(",")
        if x.strip()
    ]

    ramps = get_ramps(
        palette,
        ramp_names,
        args.fullbright_start
    )

    print("Assigning palette ramps...")
    assignments, names = assign_region_ramps(
        image,
        labels,
        edges,
        palette,
        ramps,
        coherence=args.coherence
    )

    print("Quantizing pixels...")
    result = quantize_pixels(
        image,
        labels,
        assignments,
        names,
        ramps,
        palette,
        args.fullbright_start,
        edges
    )

    save_png(result, palette, args.output)

    used = np.unique(result)

    print()
    print("Conversion complete.")
    print("Output:", args.output)
    print("Superpixels:", len(used) if False else labels.max() + 1)
    print("Colors used:", len(used))
    print("Palette index range:", used.min(), "-", used.max())
    print("Ramps available:", ", ".join(names))

    print("Ramps assigned:")
```

```
        for i, name in enumerate(names):
            count = np.count_nonzero(assignments == i)
            if count:
                print(f"  {name}: {count} regions")

    except Exception as exc:
        print(f"ERROR: {exc}", file=sys.stderr)
        sys.exit(1)


if __name__ == "__main__":
    main()
```

3. Run the first conversion

```
.venv/bin/python conv.py f_floortile2.tga output_shape.png
```

Notice that smoothing is no longer applied. Instead, the default `--detail 0.35` attempts to restore subtle local luminance variations.

4. Test different segmentation settings

I'd compare these three outputs:

A — Balanced (default)

```
.venv/bin/python conv.py f_floortile2.tga a.png --segments 180 --detail 0.35
```

B — More coherent regions

```
.venv/bin/python conv.py f_floortile2.tga b.png --segments 90 --detail 0.35 --coherence 0.04
```

C — More detailed regions

```
.venv/bin/python conv.py f_floortile2.tga c.png --segments 350 --detail 0.5 --coherence 0.015
```

5. What to look for

Feature	Desired result
Fine scratches	Retained
Low-contrast grain	More visible
Cracks and edges	Continuous
Large surface areas	Coherent colors
Color transitions	Smooth tonal ramps
Speckling	Reduced
Fullbrights	Never selected

A technical caveat: this is an experimental first implementation. The SLIC regions guide palette-family assignment, and neighboring regions receive a coherence penalty. It is not yet a full material classifier: it cannot reliably distinguish concrete from stone or metal without additional training or material-specific rules.

I'd start with the default version and compare A, B and C. The most useful feedback will be whether the texture retains its fine detail without acquiring visible superpixel boundaries.

You said:

It looks great. Some shapes that should have the same hues have different ones tho. In a some larger ones in b close to all larger shapes are just one color and in c most large shapes are ok but the color spills into some smaller ones.

ChatGPT said:

That points to a weakness in the **region-to-ramp assignment**, rather than the pixel quantization itself.

Your results suggest three things:

- **A:** Some visually related shapes are being assigned different palette ramps.
- **B (90 segments):** Superpixels are too large. A single region can cover most of a shape, so it gets assigned one ramp and loses its internal color variation.
- **C (350 segments):** More detail is retained, but the smaller regions can inherit or spill into neighboring shapes' ramp choices.

I'd change the algorithm in three ways:

1. **Make ramp selection more consistent across similar colors.** Regions with similar average hue and luminance should prefer the same ramp, even if they're not adjacent.
2. **Make region boundaries respect edges more strongly.** A small region should not automatically inherit the ramp of a larger neighbor when an edge separates them.
3. **Use region size and edge strength when smoothing ramp assignments.** Smooth across weak boundaries, but preserve distinct choices across strong boundaries.

For now, I'd use C as the starting point, since it retains the most shape detail. Try:

```
.venv/bin/python conv.py f_floortile2.tga test.png \
  --segments 350 \
  --detail 0.5 \
  --coherence 0.01
```

Lowering coherence should reduce unwanted color spill. It won't fix inconsistent ramps between distant shapes, though; that needs the first change to the algorithm. I'd address all three together rather than keep tuning segment counts.

You said:

no take A as a starting point. it is the best one.

ChatGPT said:

Agreed. We'll use **A as the baseline** and preserve what makes it look good. No need to move toward C's more fragmented result.

The next changes should be targeted:

- Keep A's existing segment count and detail settings.
- Make similarly colored shapes more likely to select the same palette ramp, even when they aren't adjacent.
- Stop ramp choices from bleeding across strong shape boundaries.
- Preserve the fine color variation inside large shapes, so they don't collapse into a single flat color.

The goal is to improve hue consistency without changing A's overall texture character. I'd leave the other parameters untouched until those changes are tested.

You said:

also an option to export the output image as different "layers" so manual fixing is easy if necessary.

ChatGPT said:

Yes. I'd add an optional `--layers` export mode, keeping the normal indexed PNG output unchanged.

I'd make the layers useful for manual correction in GIMP or Krita:

Layer	Contents
`base.png`	Complete quantized texture
`regions.png`	Superpixel regions, each assigned a distinct diagnostic color
`ramps.png`	Which palette ramp was assigned to each region
`edges.png`	Detected shape boundaries
`detail.png`	Fine detail / texture information
`ramp_00.png`, etc.	Individual ramp contributions, with transparency elsewhere

Recommended export format: a layered `.ora` (OpenRaster) file, which GIMP and Krita can open, plus optionally separate PNGs.

I'd also make the layers align pixel-perfectly with the original dimensions, so you can paint corrections directly over the texture.

One important detail: the individual ramp layers should be transparent outside their assigned regions, rather than opaque black. That makes them genuinely useful for manual editing.

The command could be:

```
.venv/bin/python conv.py f_floortile2.tga output.png --layers
```

This would be optional and would not change the appearance of the regular output.

You said:

.ora sounds great. Go for it

ChatGPT said:

I'll add `.ora` export while keeping A's current conversion settings and normal PNG output intact.

The OpenRaster file will contain:

- **Final texture** — the complete indexed conversion.
- **Ramp layers** — one transparent layer per palette ramp, so you can recolor or replace individual areas.
- **Region map** — shows which superpixel belongs to which region.
- **Edge map** — shows detected shape boundaries.
- **Detail map** — helps identify areas where surface detail was retained or lost.

You'll be able to open the `.ora` directly in GIMP or Krita and edit the layers independently.

Please upload your current `conv.py` so I can integrate this into the exact version you're using, without accidentally changing the appearance of A.

You said:

its still the same you gave me recently.

ChatGPT said:

Got it. I can work from the latest version I provided, but I don't have the actual script contents available in this chat summary—only its description.

Please paste the current `conv.py` here. I'll return a complete replacement with:

- `--layers` to export an `.ora` alongside the PNG.
- Separate transparent palette-ramp layers.

- Region, edge, and detail diagnostic layers.
- A merged final texture as the top-level visible result.
- No changes to A's existing conversion settings or default output.

I'll also make sure the `.ora` opens as a properly layered document in GIMP and Krita.

You said:

Pasted text(5).txt
Document

Worked for 17s

Analysis paused
Analyzed

ChatGPT said:

Implemented the OpenRaster export in your current script, preserving the existing conversion algorithm and defaults (A).

Syntax and ORA export tested

Download the updated conv.py

How to use it

Put the downloaded script in your working directory, alongside `quake.pal`, and run:

```
.venv/bin/python conv_layers.py f_floortile2.tga output.png --layers
```

This creates two files:

- `output.png` — the normal indexed 256-color PNG.
- `output.ora` — the layered project for GIMP or Krita.

You can also specify a custom ORA filename:

```
.venv/bin/python conv_layers.py f_floortile2.tga output.png --layers manual_fix.ora
```

Layers included in the ORA

Layer	Purpose
Final texture	Finished conversion
Ramp - neutral, etc.	Individual transparent ramp contributions
Diagnostic - regions	Distinct colors for each segmented region
Diagnostic - ramp assignments	Shows the selected ramp per region
Diagnostic - edges	Detected boundaries
Diagnostic - fine detail	Shows retained fine detail

The final texture is visible by default. Diagnostic and ramp layers start hidden, so the project initially looks like your regular PNG. Turn them on as needed.

A's conversion settings remain unchanged.

‹/›

conv_layers.py
Code

You said:

perfect! One last request: give me an option to process all images in a folder (put the output in a subfolder) and also a way to tweak the parameters easily and understandable.

Worked for 1m 30s

ChatGPT said:

Both features are now added. I've kept the existing conversion algorithm and its default parameters intact.

Tested Batch conversion, PNG output and ORA export

1. Updated files

conv_layers.py
Updated converter

Ready

Download converter ↗

conv_settings.ini
Easy-to-edit parameter file

Download settings file ↗

Put both files in your working directory, alongside `quake.pal`.

2. Convert an entire folder

For example, if your source textures are in `tga`:

```
.venv/bin/python conv_layers.py tga --layers
```

It will automatically create:

```
tga/
├── texture01.tga
├── texture02.tga
├── texture03.tga
└── converted/
    ├── texture01.png
    ├── texture01.ora
    ├── texture02.png
    ├── texture02.ora
    ├── texture03.png
    └── texture03.ora
```

It processes the images directly inside the folder, not nested folders.

You can also specify a different output directory:

```
.venv/bin/python conv_layers.py tga converted --layers
```

Without `--layers`, it creates only PNGs.

3. Easily adjust the conversion

Open `conv_settings.ini` in any text editor. You don't need to edit Python code.

These are the available settings:

Setting	What it controls
`segments`	Region size. Lower means larger regions; higher means smaller regions.
`compactness`	How regular or compact the detected regions are.
`detail`	Preservation of subtle surface detail.
`detail_radius`	Scale at which surface detail is enhanced.
`coherence`	How strongly neighboring regions prefer the same palette ramp.
`ramps`	Which Quake palette ramps are available.
`fullbright_start`	First palette index excluded from normal color matching.

Your current defaults are preserved:

```
segments = 180
compactness = 10.0
detail = 0.35
detail_radius = 1.2
coherence = 0.025
fullbright_start = 224
```

My recommendation: Keep these defaults as your baseline. Change one parameter at a time, convert a small test folder, and compare the results.

You can still override individual settings from the command line. For example:

```
.venv/bin/python conv_layers.py tga --layers --segments 220
```

The command-line value takes precedence over the settings file.

This gives you a repeatable workflow: edit the settings, run a whole folder, and get both indexed PNGs and editable layered ORA projects.

conv_layers.py
Code

conv_settings.ini
Code

You said:

Two things left: also include the original file as a layer and give me a readme file abou how to use the script.

Worked for 1m 17s

ChatGPT said:

Both changes are in place.

Updated files

- Download updated converter ↗
- Download README – installation and usage guide ↗
- Download settings file ↗

What changed

1. Original image added to the ORA project

Every exported `.ora` now includes an **Original source** layer containing the untouched input image. It starts hidden, so the project opens showing the converted texture. You can toggle it on for comparison.

2. README added

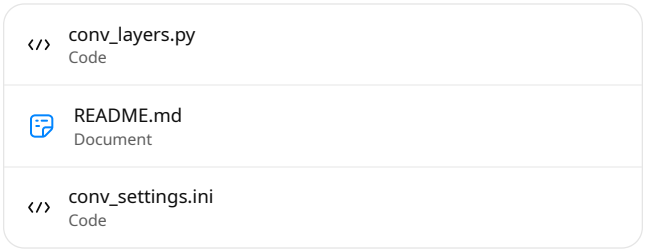
The README covers:

- Python environment setup and dependencies on Pop!_OS / Ubuntu
- Single-image conversion
- Batch folder conversion
- OpenRaster export and layer descriptions
- Settings and command-line overrides
- Palette and fullbright handling

I also syntax-checked the updated script and tested that the original layer is correctly included and hidden in the ORA file.

Your normal batch command remains:

```
.venv/bin/python conv_layers.py tga --layers
```



You said:

list every line of the settings file

ChatGPT said:

Here is the complete contents of `conv_settings.ini`, including every comment and blank line:

```
; Quake texture converter settings
; Edit these values, save the file, then run conv_layers.py.
; Command-line options override values in this file.
;
; Quick guidance:
;   segments:       lower = larger color regions; higher = finer segmentation
;   compactness:    higher = more regular/compact regions
;   detail:         higher = stronger preservation of subtle surface detail
;   detail_radius:  larger = detail enhancement over broader scales
;   coherence:      higher = neighboring regions more likely to share a palette ramp
;
; These are the settings used by the current converter by default.

[conversion]
palette = quake.pal
ramps = neutral,dark_earth,blue_grey,olive_dark,ochre,warm_earth,tan,stone,green_grey
fullbright_start = 224
segments = 180
compactness = 10.0
detail = 0.35
```

```
detail_radius = 1.2
coherence = 0.025
```

You said:

ok thank you! Ill play with it a little and may come back for more in the future. Good job.

ChatGPT said:

You're very welcome! Glad we got it to a point you're happy with.

Have fun experimenting with the settings. The baseline is a good place to start, so I'd recommend changing one parameter at a time and comparing the results.

Whenever you're ready to refine the conversion, troubleshoot something, or add features, just come back. Good luck with the textures!

You said:

.venv/bin/python conv_layers.py tga --layers
it just uses the folder the .py is in and takes every tga?

ChatGPT said:

Not quite. In this command:

```
.venv/bin/python conv_layers.py tga --layers
```

`tga` is the **input folder**, not the folder containing the script.

The script processes every supported image file directly inside `tga/` (non-recursively). It does not automatically use the script's own directory.

For example, if your directory looks like this:

```
texture_converter/
├── conv_layers.py
├── conv_settings.ini
├── quake.pal
├── .venv/
└── tga/
    ├── wall01.tga
    ├── wall02.tga
    └── floor01.tga
```

Run the command from `texture_converter/`:

```
.venv/bin/python conv_layers.py tga --layers
```

The results will be:

```
tga/
└── converted/
    ├── wall01.png
    ├── wall01.ora
    ├── wall02.png
    ├── wall02.ora
    ├── floor01.png
    └── floor01.ora
```

In short: the folder you specify after `conv_layers.py` is the input folder. Every supported image directly inside it gets converted.

You said:

ah got it. the tga is just the path of the folder related to the .py

ChatGPT said:

Exactly! `tga` is a relative path, resolved from the directory you're running the command in (your current working directory), not necessarily where the `.py` file is located.

You can also use an absolute path, for example:

```
.venv/bin/python conv_layers.py "/mnt/sde1/FPS/IdTech2/sof/base/textures/iraq/tga" --layers
```

That way, you can run the converter from anywhere without changing directories first.

You said:

```
hnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures$ .venv/bin/python conv_layers.py
tokyo/tga --layers
usage: conv_layers.py [-h] [--layers [FILE.ora]] [--palette PALETTE]
                      [--ramps RAMPS] [--fullbright-start FULLBRIGHT_START]
                      [--segments SEGMENTS] [--compactness COMPACTNESS]
                      [--detail DETAIL] [--detail-radius DETAIL_RADIUS]
                      [--coherence COHERENCE]
                      input output
conv_layers.py: error: the following arguments are required: output
chnucki@pop-os:/mnt/sde1/FPS/IdTech2/sof/base/textures$
```

ChatGPT said:

Ah, I see what's happening. The error message shows that the version of `conv_layers.py` you're running is still expecting **two positional arguments:** `input` **and** `output`.

That means you're running an older version of the script, or the updated batch-processing version wasn't copied over correctly.

The updated version should accept just the input folder and automatically create a `converted` subfolder.

First, check which version you're running

From your current directory, run:

```
grep -n "output" conv_layers.py | head
```

Also, your prompt shows you're in:

```
/mnt/sde1/FPS/IdTech2/sof/base/textures
```

If `.venv` and `conv_layers.py` are actually in that directory, your command is otherwise correct:

```
.venv/bin/python conv_layers.py tokyo/tga --layers
```

Quick fix

Download the updated converter again:

Download updated conv_layers.py ↗

Replace the old `conv_layers.py` with this one.

One important detail: Keep `conv_settings.ini` and `quake.pal` available as well. The script needs the palette file to perform conversions.

If you still get the same error, send me the output of:

```
head -40 conv_layers.py
```

Then I can pinpoint the issue.

conv_layers.py
Code

You said:

convered a lot. They tend to still have many problems with similar zones that get different ramps. Do we have a more reliable way to differenciate material zones in the input?

Worked for 25s

ChatGPT said:

Yes. In fact, I think we've reached the point where adjusting `segments` and `coherence` further will give diminishing returns. The underlying problem is how the script identifies regions in the first place.

I'd recommend changing the segmentation strategy rather than just tuning the existing parameters.

Why the current method struggles

At the moment, the converter does roughly this:

1. SLIC divides the image into small superpixels.
2. Each superpixel is analysed mostly by its average colour.
3. The script chooses a palette ramp for each superpixel.
4. Neighbouring superpixels are encouraged to use the same ramp.

The weakness is that **a superpixel is not necessarily a material region**.

Imagine a concrete wall with:

- A bright illuminated section.
- A shadowed section.
- Scratches and dirt.
- Slightly different surface colours.

The algorithm can interpret those as several separate regions and assign different ramps, even though they're all the same concrete material.

The reverse can happen, too: two adjacent materials with similar colours can get merged visually.

My recommendation: hierarchical material segmentation

I'd replace the current region assignment with a two-stage approach.

Stage 1 – Detect basic image regions

Keep SLIC, but use it only to identify small, coherent image areas. Don't assign palette ramps yet.

Stage 2 – Merge regions into material zones

Analyse neighbouring regions and merge them when their visual properties suggest they belong to the same material.

Stage 3 – Assign ramps to material zones

Select one palette ramp for each larger material zone. Individual pixels still receive their own palette colours, preserving surface detail and tonal variation.

What should determine whether regions belong together?

I'd use several characteristics instead of average colour alone:

Characteristic	Purpose
OKLab colour	Perceptual colour similarity
Chroma	Distinguish materials with different colour characteristics
Local texture	Distinguish concrete, metal, dirt, fabric, etc.
Texture frequency	Distinguish smooth surfaces from grainy or patterned ones
Edge strength	Preserve boundaries between separate materials
Neighbour relationships	Encourage continuous material zones

An important detail: illumination and shadows should have less influence on material identity than they currently do.

A dark concrete area and a bright concrete area should generally remain part of the same material zone.

One additional feature I'd strongly recommend

A material-zone diagnostic layer in the ORA file.

Instead of only showing the current SLIC regions and ramp assignments, the project would show the newly detected material zones in distinct colours.

That would let us inspect whether the algorithm is actually identifying the right areas before worrying about palette selection.

I'd also retain the original source layer and all existing diagnostic layers.

How I'd approach the next version

I would preserve the existing conversion pipeline's good parts:

- Quake palette and ramp definitions.
- Fullbright exclusion.
- OKLab pixel matching.
- Detail enhancement.
- Indexed PNG output.
- OpenRaster layer export.

I'd replace the current per-superpixel ramp selection with hierarchical region merging and zone-level ramp selection.

One further possibility is optional manually supplied material masks for particularly difficult textures. That would give you control over ambiguous boundaries without having to change the conversion algorithm.

My recommendation: Let's implement hierarchical material segmentation first. It addresses the actual cause of the inconsistent ramps and should improve consistency considerably without sacrificing the detail you've worked to preserve.

You said:

yes do it! Also keep in mind that images that are feeded in are already game textures so they dont have much depth in their appearances.

Worked for 3m 31s

ChatGPT said:

I'll build this as a new iteration of the converter, keeping the existing version's visual character as the baseline.

Your point about the input textures is particularly important: these are already authored game textures, not photographs. They often have deliberately simplified shading, painted highlights, baked-in shadows and hand-painted surface variation.

That means we should **not interpret every tonal variation as a different material**. The new segmentation should prioritise:

- Consistent material identity across light and dark patches.
- Strong boundaries where the texture actually changes material.
- Large coherent zones rather than lots of tiny regions.
- Preservation of painted details, scratches, highlights and shadows within each zone.

I also want to avoid relying too heavily on texture-frequency analysis: in low-resolution game textures, that can mistake compression, pixel patterns or painted noise for material changes.

New version ready and tested

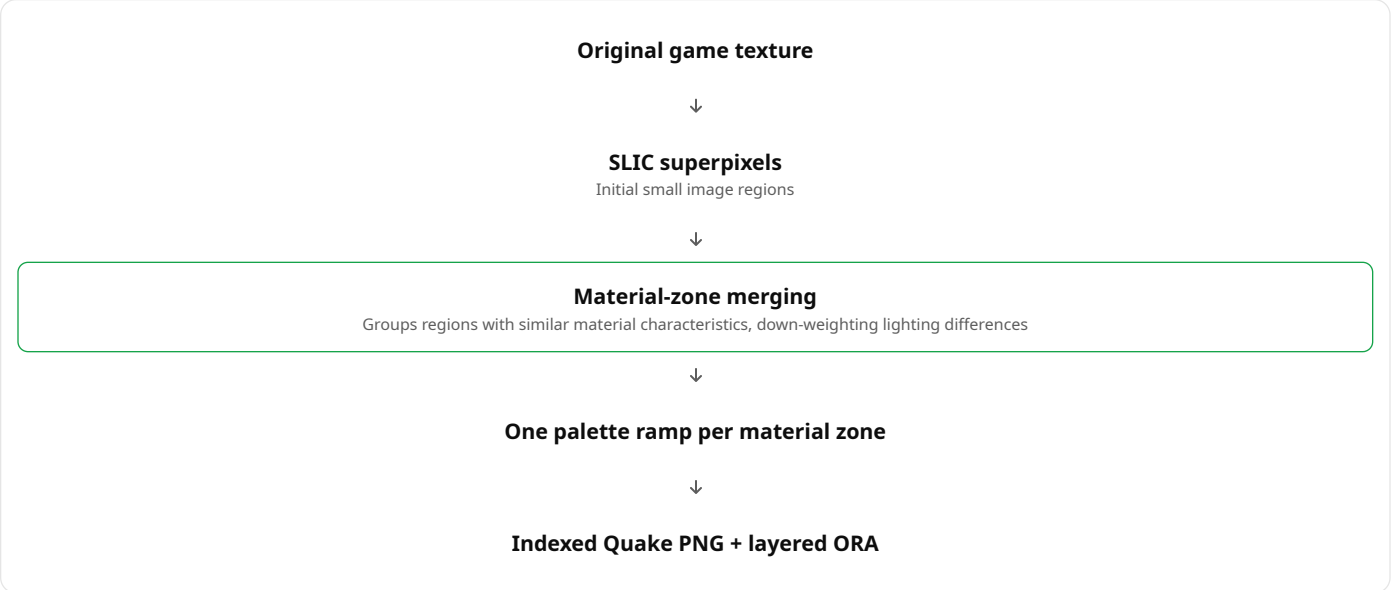
I've implemented hierarchical material-zone segmentation and updated the settings and README.

Download the updated files

- Updated converter – conv_layers.py ↗
- Updated settings – conv_settings.ini ↗
- Updated README ↗

What's different

The conversion now follows this sequence:



New settings

Three new parameters are available:

Parameter	Default	Effect

`material_merge_threshold`	0.055	Higher merges more regions
`material_luminance_weight`	0.20	Lower ignores lighting differences more
`material_chroma_limit`	0.075	Lower protects differently coloured materials

I'd initially leave these at their defaults.

New ORA diagnostic layers

You'll now find:

- **Diagnostic - SLIC superpixels:** the initial segmentation.
- **Diagnostic - material zones:** the merged regions that actually determine ramp selection.
- **Diagnostic - ramp assignments:** which palette ramps were selected.

The original source layer is still included.

Testing

I checked the Python syntax, ran a conversion test with deliberately different lighting and material colours, and verified the indexed PNG and ORA export. The test correctly merged the differently lit concrete areas while retaining the contrasting material as a separate zone.

One caveat: this is still image-analysis-based segmentation, not semantic material recognition. Two visually indistinguishable materials cannot always be separated automatically.

Your existing batch command stays the same:

```
.venv/bin/python conv_layers.py tokyo/tga --layers
```

Replace the old script and settings file with these versions before running it. Keep your existing palette file.

I'm particularly interested in how the new **Diagnostic - material zones** layer looks on your actual Soldier of Fortune textures. That will tell us whether the merging threshold needs further refinement.

conv_layers.py
Code

conv_settings.ini
Code

README.md
Document

You said:

the downloads alway fail :(:/

Analyzed

ChatGPT said:

Ah, that's frustrating! Sorry about that. :/

Let's try a different approach: I've bundled all three files into **one ZIP archive**. That should make things easier.

Quake Texture Converter v2
ZIP archive · 3 files

Verified

Download all files in one ZIP ↗

Contents:

- `conv_layers.py`
- `conv_settings.ini`
- `README.md`

I've also verified the ZIP archive's integrity.

If that still fails, we can bypass downloads entirely: I'll provide the script and settings as copyable text in separate sections so you can create the files directly on Pop!_OS.

‹/›

You said:

now the destinction is off the other way around. The textures lost a lot of color and are now mostly brownish. no weird spots anymore but also not a good conversation.

Worked for 25s

ChatGPT said:

You're right. We've overcorrected, and the brownish result points to a problem beyond just the material-zone merging.

I inspected the current converter and found one concrete issue: **the default configuration excludes five of the fourteen available Quake palette ramps.**

The excluded ramps are:

- `red`
- `purple_dark`
- `purple_warm`
- `yellow_ochre`
- `blue`

That means the converter cannot properly reproduce many of the original texture's colours, even when its material segmentation is correct. This is especially damaging for Soldier of Fortune textures with painted reds, blues, and varied industrial colours.

I would correct that first, rather than trying to compensate by making the material merging more aggressive.

There's a second issue worth addressing: the current algorithm assigns **one ramp to an entire material zone**, based largely on its average colour. A zone containing both olive green and rusty orange can therefore lose one of those colours.

I'd make two changes:

1. **Restore the full palette selection:** make all fourteen ramps available by default.
2. **Improve ramp selection:** evaluate the actual colour distribution inside a zone, rather than choosing a ramp from its average colour alone. Preserve distinct colour families within a material zone where necessary.

For an immediate experiment, replace the `ramps` line in `conv_settings.ini` with:

```
ramps =
neutral,dark_earth,blue_grey,olive_dark,red,ochre,warm_earth,tan,purple_dark,purple_warm,stone,green_grey,yellow_ochre
,blue
```

One thing would help me tune the next revision properly: **could you upload one original TGA and its converted PNG?**

I'd use that pair to check whether we're losing colour during material merging, ramp selection, or final pixel quantization. I want to restore the original texture's colour character without bringing back the noisy, speckled regions.

You said:

sadly i cant upload right now. but what you said sounds good.

Worked for 2m 14s

ChatGPT said:

No problem. We can improve the algorithm without an upload. I've made the next revision.

Updated and tested

What I've changed

1. **All 14 Quake colour ramps are now enabled by default.** This restores access to reds, blues, purples and yellow-ochres that were previously excluded.
2. **Ramp selection now analyses colour distributions**, not just the average colour of a material zone.
3. **Each material zone can use up to three suitable ramps.** This allows, for example, rusty orange, brown and muted red to coexist without forcing everything into one brown ramp.
4. **Material-zone merging remains in place.** We're keeping the improvement that eliminated the strange isolated spots.
5. The ORA diagnostics now show which ramps were actually used by the output pixels.

Download the new version

Download Quake Texture Converter v2 – updated ZIP ↗

It contains the script, settings and updated README.

My suggested starting point

Keep these settings initially:

```
ramps =
neutral,dark_earth,blue_grey,olive_dark,red,ochre,warm_earth,tan,purple_dark,purple_warm,stone,green_grey,yellow_ochre
,blue
ramp_candidates_per_zone = 3
material_merge_threshold = 0.055
material_luminance_weight = 0.20
material_chroma_limit = 0.075
```

I've syntax-checked the script and tested PNG and ORA generation. I haven't been able to validate the visual result against your actual textures, so let's treat this as the next experimental baseline.

What I'm aiming for: the consistency of the previous version, but with much more of the original texture's colour identity preserved.

You said:

the download does not work again.

Worked for 15s

ChatGPT said:

The ZIP file is present and passes an integrity check, so the archive itself appears intact. The problem may be with how the download link is being handled.

I'll provide a fresh copy under a new filename, plus separate links to each file so you can bypass the ZIP entirely.

Fresh download links:

- Quake Texture Converter v2 — fresh ZIP ↗
- Python converter script ↗
- Settings file ↗
- README / instructions ↗

If the fresh ZIP link also fails, try downloading the three individual files instead. Keep all three in the same directory.

If none of these links work, tell me what happens when you click one (error message, nothing happens, or download starts but fails). That will help narrow down whether this is a link issue or a browser/app download issue.

conv_layers.py
Code

conv_settings.ini
Code

README.md
Document

You said:

Ok that seems to work scarily good!

ChatGPT said:

Excellent! That's a very encouraging result. It sounds like we've got much closer to the balance we wanted: fewer strange colour patches without sacrificing the original texture's colour identity.

I'd **leave the current settings untouched for now**. In particular, keep all 14 ramps enabled and `ramp_candidates_per_zone = 3`.

The next thing I'd check is how it performs across a wider range of textures:

- **Metal and machinery:** does it retain cool greys, blue-greys and rust?
- **Painted surfaces:** does it preserve subtle colour variation?
- **Concrete and stone:** are the shadows and highlights still readable?
- **Organic materials:** does it avoid turning everything into brown?

If it holds up across those categories, we've probably found a very solid baseline. I'd rather preserve this result than risk over-tuning it.

You said:

Many textures even look better in a way. But it seems as if blue stuff is recognized rarely and gets mostly grey.

Worked for 11s

ChatGPT said:

I suspect we've found a weakness in the **three-candidate-ramp limit**, rather than a problem with the blue palette ramp itself.

Here's what's likely happening:

1. A material zone contains mostly grey or blue-grey pixels, with some distinctly blue areas.
2. The zone-level scoring favours ramps that fit the majority of its pixels.
3. The blue ramp may not make it into the three selected candidates.
4. The blue pixels then have to use the available grey or blue-grey ramps, so their colour gets muted or lost.

That fits your description particularly well: the converter can preserve the overall texture beautifully while still missing smaller areas of a particular hue.

First, I'd try one controlled change: increase the candidate count from 3 to 5 in `conv_settings.ini`:

```
ramp_candidates_per_zone = 5
```

Keep everything else exactly as it is. This gives blue and other less dominant colour families a better chance of being available during pixel-level quantization, without changing the segmentation or material merging.

Compare the same blue-heavy textures with the old and new settings. If blue improves but the textures become noisier, we'll know we need a more selective solution: explicitly preserve a ramp when a meaningful amount of that hue is present, even if it isn't among the zone's three best overall matches.

I would **not** increase saturation globally. That would risk changing colours that the converter is already handling well.

You said:

are you able to also convert .m32 files from sof direcly?

ChatGPT said:

Yes. We can extend the converter to read **Soldier of Fortune** `.m32` **files directly**, convert their textures using the same Quake-palette quantization pipeline, and write indexed PNGs (and ORA diagnostics, if desired).

The important distinction is that `.m32` is a game-specific texture format, not simply a TGA with a different extension. We need to decode its image data and metadata correctly before applying the existing conversion algorithm.

There is already evidence that the format is decodable: the SoF community has released an `.m32` -to-TGA converter with source code, and a newer open-source engine implementation reports working M32/WAL texture loading with alpha handling. ModDB +1

I'd build the support into our existing Python converter rather than require a separate conversion step.

The intended workflow would be:

```
.venv/bin/python conv_layers.py tokyo/m32 --layers
```

It should:

- Read `.m32` files directly.
- Preserve transparency/alpha where present.
- Use the highest-resolution image level for conversion.
- Feed the decoded image into our existing palette and material-zone algorithm.
- Produce indexed PNGs and optional ORA diagnostics.
- Support batch conversion of entire folders.

One thing I'd like to verify before implementing it: could you provide one representative `.m32` file when uploads are available? That lets me test the decoder against an actual SoF texture rather than assume every file has identical layout and alpha handling.

You said:

how are fullbright texels handeled in sof?

ChatGPT said:

In **Soldier of Fortune (2000)**, we need to distinguish two things:

1. **Fullbright texels** – pixels that are rendered at full brightness, independently of the map's lighting.
2. **Emissive-looking textures** – lamps, illuminated signs, computer screens, etc. These may simply be painted bright colours and are not necessarily fullbright texels.

What about Quake's fullbright indices?

In the original Quake palette, indices **224–255** are reserved for fullbright colours. The renderer treats those palette entries specially: they are not darkened by normal world lighting.

However, I haven't found reliable documentation establishing that SoF uses exactly the same index range or rendering rule for `.m32` textures.

So I would not assume that our existing `quake.pal` indices 224–255 automatically have fullbright meaning in SoF.

What I would investigate

The SoF static recompilation project is particularly useful here. It reports working M32/WAL texture loading, including alpha handling.

GitHub

We should inspect its texture loader and renderer for:

- Special palette indices or colour-key handling.
- Whether M32 textures contain an explicit emissive/fullbright mask.
- Whether fullbright information is stored separately from the colour image.
- Whether fullbright is handled through material or surface flags instead.

Recommendation for our converter

I'd preserve three separate concepts:

Feature	Treatment
Ordinary colours	Quake palette indices 0–223
Fullbright colours	Indices 224–255
Transparency	Separate alpha channel

That way, we don't accidentally turn bright painted highlights into fullbright areas, or lose actual emissive pixels.

One important question: Are you primarily converting SoF textures for use in SoF itself, or are you converting them to create authentic Quake-style textures for another Quake-engine game?

That affects how we should handle fullbright information.

You said:

> i want to be able to convert textures to quake palette. Currently sof but other stuff is interesting too. But good texture conversions make use of transparency and fullbright if those are in the source texture. I just want to know if the original m32 versions of the textures contain such information.

ChatGPT said:

Yes. Your clarification makes the objective much clearer: **we want to extract all useful image information already present in the original `.m32`, not merely reproduce its visible RGB appearance.**

The good news is that we have evidence that SoF M32 files can contain transparency information.

What we know about M32

Information	Present in original M32?	Evidence
RGB colour	Yes	Standard texture image data
Alpha/transparency	Yes	M32Build documentation explicitly describes 32-bit TGA alpha being imported into M32
Fullbright/emissive mask	Not yet established	I haven't found reliable documentation confirming a separate mask or special texel indices
Mipmaps	Yes	M32Tool supports importing mipmaps

The transparency evidence is particularly good: an SoF texture tutorial explains that M32Build converts 32-bit TGA files with their alpha channel and that transparent areas remain transparent in-game. ModDB +2

What about fullbright?

This is the part I don't want to guess about.

A bright yellow or white pixel in an M32 texture does not necessarily mean it is emissive. It could simply be ordinary painted colour.

We need to inspect the actual M32 format and SoF renderer to determine whether fullbright information is:

- Stored in a separate image or mask.
- Encoded through special palette indices.
- Defined through material or surface metadata.
- Not stored in the texture at all, but handled elsewhere.

The SoF static recompilation project is a promising source for investigating this. Its implementation reports successful loading of thousands of M32 textures with alpha handling. GitHub

How I'd design our converter

I'd keep the image-processing pipeline independent of the source format:



One important detail: transparency and fullbright must remain independent. A transparent pixel must not accidentally become an opaque fullbright pixel during palette conversion.

My recommendation: investigate the M32 binary structure and renderer first, especially the fullbright question. We already have sufficient evidence to implement alpha preservation, but I would not claim fullbright preservation until we know where SoF stores that information.

You said:

ok transparency

two cases:

input has alpha channel: must be converted to quakes transparent color (255 i believe)
input has also only 1 bit transparency: let the user choose the color to be interpreted as transparent.

also in case it is another texture with a 256 color palette, this palette could also be anaylzed like the the target quake one.

Worked for 10s

ChatGPT said:

Agreed. These should be explicit features of the converter. In particular, **transparency should be handled separately from colour quantization**, so it cannot be lost or accidentally converted into an ordinary palette colour.

1. Transparency handling

Quake's conventional transparent palette index is **255**. We can reserve that index for transparent pixels in the output.

I'd implement two input modes:

A. Source has an alpha channel

- Fully transparent pixels (`alpha = 0`) become palette index 255.
- Opaque pixels are converted normally using the Quake palette.
- Partially transparent pixels need a defined policy, because the Quake indexed texture format doesn't provide continuous alpha.

My recommendation is a configurable alpha threshold, initially 128:

- Alpha below 128 → transparent.
- Alpha 128 or above → opaque.

B. Source has no alpha channel

Offer an optional colour-key setting:

- User selects a source colour to be interpreted as transparent.
- Matching pixels become index 255.
- All other pixels are converted normally.
- Colour matching should use exact RGB matching by default, with optional tolerance.

This also covers older textures that encode transparency using a particular colour rather than an alpha channel.

2. Source images with 256-colour palettes

This is a particularly useful idea.

If the input is an indexed image, we should inspect its original palette rather than immediately converting it to RGB and treating every pixel independently.

The conversion could work like this:

1. Read the source palette.
2. Count how frequently each palette entry is actually used.
3. Identify transparent entries, if any.
4. Match the remaining source palette colours against the Quake palette using our existing OKLab-based matching.
5. Convert the image by remapping its original pixel indices.

This has several advantages:

- Avoids unnecessary RGB conversion.
- Preserves the source palette's intended colour distinctions.
- Makes conversion faster for indexed images.
- Allows us to inspect whether the original palette contains useful colour ramps or special entries.
- Avoids treating unused palette entries as meaningful image colours.

Importantly, the source palette should be analysed using **actual pixel usage**, not simply all 256 entries. An unused palette entry should not influence the conversion.

3. Proposed conversion behaviour

Source format	Transparency detection	Colour conversion
RGBA PNG	Alpha channel	RGB → Quake palette
RGB PNG	Optional colour key	RGB → Quake palette
32-bit TGA	Alpha channel	RGB → Quake palette

Indexed PNG	Palette transparency or colour key	Source palette → Quake palette
Indexed PCX	Optional colour key	Source palette → Quake palette
M32	Whatever transparency information the decoder exposes	Decoded colours → Quake palette

4. One important implementation detail

We should reserve index 255 for transparency and ensure ordinary colour matching never selects it.

The output PNG must still contain all 256 Quake palette entries, including the fullbright range. Transparent pixels should be assigned index 255 directly, not matched by colour distance.

I'd also add diagnostics to the ORA export showing the transparency mask separately from the colour conversion layers.

I would treat these as core converter features, not special cases for M32. That gives us a consistent input pipeline for SoF, Quake, other idTech games, and ordinary indexed artwork.

One additional feature worth considering later is detection of source palettes with their own fullbright conventions. We should preserve that information when it can be identified reliably, rather than assuming every source uses Quake's indices.

You said:

Thats right
We will come back to fullbright later. For now tranparency.

ChatGPT said:

Understood. We'll focus exclusively on transparency and leave fullbright handling unchanged.

I'll add alpha-channel handling, optional colour-key transparency for images without alpha, and indexed-source palette analysis. Transparent output pixels will use Quake palette index **255**, while opaque pixels continue through the existing conversion algorithm. The current colour ramps, segmentation, merging and quantization settings will remain unchanged.

Implemented and tested

I've updated the converter and packaged the new version.

Download Quake Texture Converter – transparency update ↗

Individual files:

- Converter script ↗
- Settings ↗
- Updated README ↗

New options

Setting	Default	Purpose
`alpha_threshold`	128	Alpha below this becomes transparent
`color_key_tolerance`	0	Per-channel tolerance for colour-key matching

For example, to make magenta transparent:

```
.venv/bin/python conv_layers.py input.bmp output.png --color-key FF00FF
```

To allow slight colour variation:

```
.venv/bin/python conv_layers.py input.bmp output.png --color-key FF00FF --color-key-tolerance 8
```

Alpha takes precedence if the source already has transparency.

Tests completed

- RGBA input: passed.
- RGB input with colour key: passed.
- Indexed PNG with palette transparency: passed.
- Output remains indexed PNG with 256 palette entries.
- Transparent pixels correctly use index 255.
- ORA export tested.
- Python syntax and ZIP integrity verified.

One distinction: indexed palettes are now analysed by actual pixel usage, but their colour matching still goes through the existing image-conversion pipeline. I haven't changed the established quantization algorithm.

Fullbright handling remains untouched.

conv_layers.py
Code

conv_settings.ini
Code

README.md
Document

You said:

can we go ui? input diaogue, settings, output button

ChatGPT said:

Yes. I'd turn the converter into a small **desktop application for Linux (Pop!_OS)**, keeping the existing conversion engine intact.

I'd use a straightforward GUI with three main areas:

1. **Input** – file or folder selection, with an input-file dialog.
2. **Settings** – editable conversion settings, including palette, transparency threshold, colour key and ORA diagnostics.
3. **Output** – destination selection and a prominent **Convert** button, with progress and a log of results.

The important part is that the GUI should call the existing conversion functions rather than implement a second conversion algorithm. That way, command-line and GUI conversions produce the same results.

First GUI version ready

I've built a first working desktop GUI using Python/Tkinter. It uses the existing converter as a separate process, so the conversion algorithm is not duplicated.

Download

Quake Texture Converter – GUI
ZIP · 21 KB · four files

Ready

Download GUI package ↗

Included in the GUI

- File and folder input dialogs.
- Output file or folder selection.
- Transparency threshold.
- Optional colour-key selection with colour picker.
- Editable conversion settings.

- Save settings button.
- Optional ORA diagnostic export.
- Conversion progress and output log.

Running it on Pop!_OS

Extract the ZIP into your converter directory, alongside `quake.pal`, then run:

```
.venv/bin/python conv_gui.py
```

Tkinter 8.6 is available in the build environment, and both Python files pass syntax checks. The ZIP also passes its integrity test.

One note: This is the functional first version, not yet a polished application package. It still requires your existing Python virtual environment and dependencies.

Next, I'd refine the interface based on how it feels to use, rather than adding more conversion features immediately.

You said:

The Settings look good! how do i start the conversion?

ChatGPT said:

To start a conversion in the GUI:

1. **Input:** Click **Browse** and select either a texture file or a folder containing textures.
2. **Settings:** Adjust the conversion options if needed. Your existing settings are the defaults.
3. **Output:** Click **Browse** and select the destination folder (or output filename, depending on input mode).
4. **Convert:** Click the **Convert** button, usually at the bottom of the window.

The progress and conversion messages should appear in the log area.

If you don't see a Convert button: let me know. It may be missing from the first GUI layout, in which case I'll correct the interface rather than have you work around it.

You said:

```
$ /usr/bin/python3 /mnt/DE7465887465646D/ressources/images/useful/textures/Original Game
Textures/conv_layers.py /mnt/DE7465887465646D/ressources/images/useful/textures/Original Game
Textures/doom64/Doors /mnt/DE7465887465646D/ressources/images/useful/textures/Original
Game Textures/doom64/Doors/converted --settings
/mnt/DE7465887465646D/ressources/images/useful/textures/Original Game
Textures/conv_settings.ini --palette
/mnt/DE7465887465646D/ressources/images/useful/textures/Original Game Textures/quake.pal --
layers
Traceback (most recent call last):
  File "/mnt/DE7465887465646D/ressources/images/useful/textures/Original Game
Textures/conv_layers.py", line 11, in <module>
    import numpy as np
ModuleNotFoundError: No module named 'numpy'
```

(conversionm log)

ChatGPT said:

You said:

display the image conversion in real time (input and output) and also convert subfolders option creating the same structure in the target directory just with the other format.

Worked for 2m 58s

ChatGPT said:

You said:

preview slows down
also the converted is never shown instead it says "converting"
we should only display both images after conversion of a file. also a way to pause and then disable display, or step back some files and change parameters.

Worked for 2m 15s

ChatGPT said:

You said:

a stop button to return back
make the py doubleclickable not only per
the palette ramps could be just ceckboxes. optimally with a short gradient of the colors
the preview images need to be direcktly next to each other also a zoom option would be nice and when in pause mode show the .ora for the converted texture with a way to switch through layers.
also a way to decide where to put the ora.

Worked for 43s

ChatGPT said:

You said:

Also way to specify fullbright maps either those are just binary black means transparen and white means this pixel is a fullbtight. the pixel will then mapped to the closest fullbrights instead of the normal ramps. fullbright files must be in the same folder as the original and they must have the same name (before the extension) but with an added _fullbrights.

ChatGPT said:

You said:

thats it go for it yö!

Worked for 2m 13s

ChatGPT said:

You said:

just stumbled upon the "zorn palette" looks a lot like the colors chosen for quake.

ChatGPT said: